Process and Threads

Lecture# 2

Process

- Definitions:
  - A process is a program in execution
  - The entity that can be assigned to and executed on a processor
- Processes are a fundamental concept of an operating system
  - As a unit of execution, it enables the concurrent execution of multiple programs on a system
- CPU switches between processes
  - All the running software on a computer system, including the operating system itself, is organized as a set of processes

Process

- A process consists of three basic components
  - An executable program code
  - Associated data needed by the program
  - Execution context
- Information needed by the operating system to manage a process
  - Process ID
  - CPU registers, Program counter
  - Stack content
- Execution context essential for process switching
- Operating system maintains a process list of all processes currently in the system

Process Control Structures

- Process Table: holds one entry per process
  - Process Control Block
- Process Control Block
  - Describes a single process and its current state
- Process Image
  - Memory allocation for process

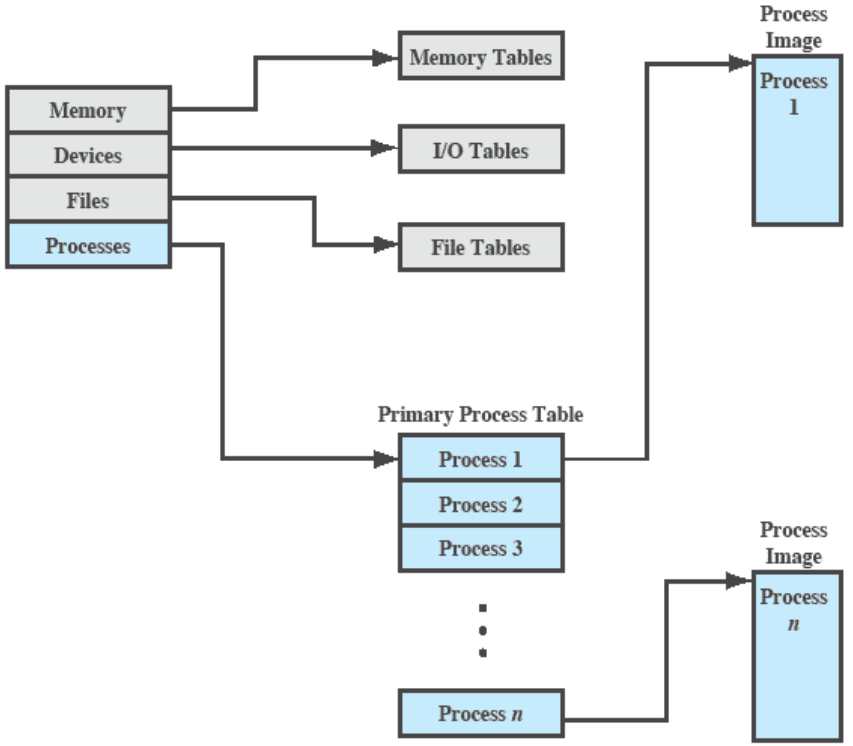


Figure 3.11 General Structure of Operating System Control Tables

Process Control Block

Process Identifier	A unique identifier associated with the process
Process State	The state of the process, when it was interrupted
Priority	Priority level relative to other processes
Program counter	Address of next instruction to be executed
Memory pointers	Pointers to program code and data
Context data	Register content
I/O status information	Outstanding I/O requests, assigned I/O devices, list of open files
Accounting information	Used processor time, time limits etc.
. . .	

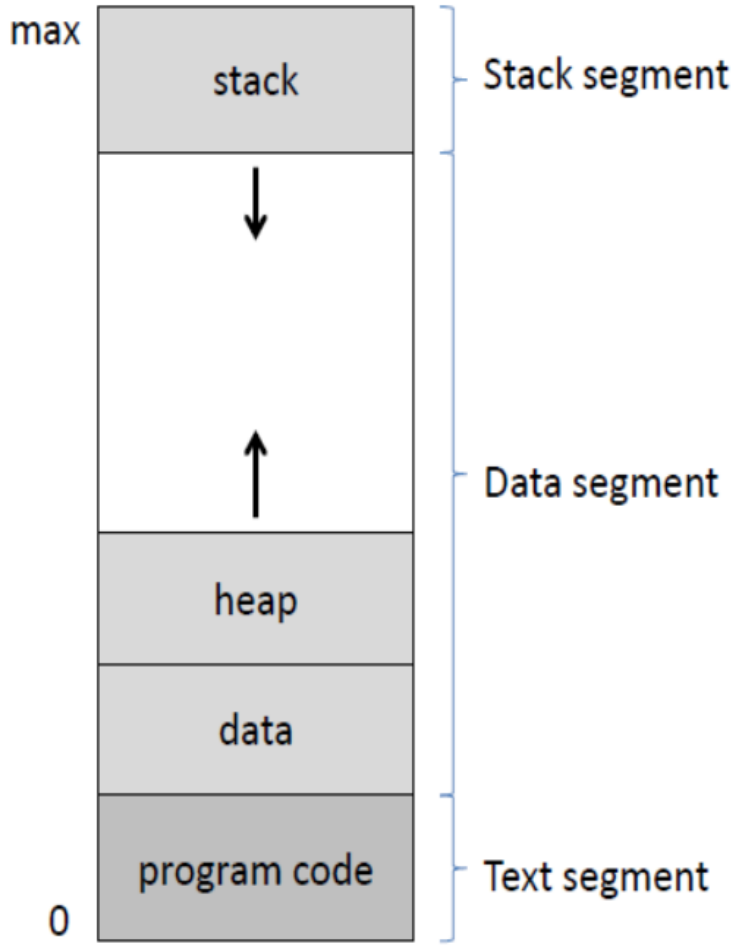
The Process Control Block (PCB) contains all the information necessary to suspend a process and later resume its execution

Process Control Block

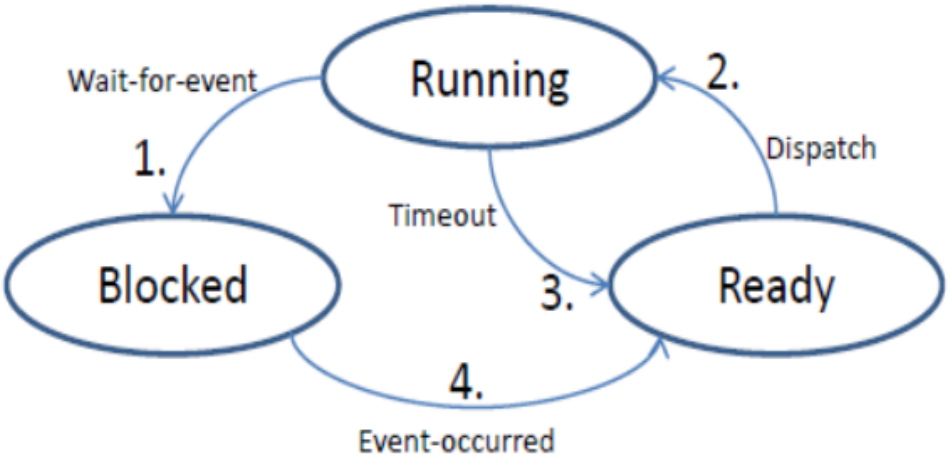
- Process identification
  - Process ID
  - Identifier of creating process and user
- Processor state information
  - CPU registers: user-visible registers, control and status registers, stack pointers, program counter
- Process control information
  - CPU scheduling information: process state, priority, scheduling-related information
  - Process privileges: access privileges to memory, allowed instructions
  - Memory management information: information about the virtual memory assigned to this process (segment, paging tables, various control registers)
  - Accounting information: time limits, accumulated CPU time etc.
  - I/O status information: resource ownership, open files, allocated I/O devices

Process Image

- Allocation of a process in memory
  - Process Image
- Each process runs in its own virtual memory address space that consists of
  - Stack segment: used for function and system calls
  - Data segment: variables, both static and dynamic allocation on the heap
  - Text segment: the program code, usually read-only
- Invoking the same program multiple times results in the creation of separate processes, each with its distinct virtual address space and environment



Process States



1. Process blocked for I/O
2. Dispatcher schedules another process
3. Dispatcher schedules this process
4. Input becomes available, blocked process made ready

We can distinguish three basic process states

- Running: actually using the CPU
- Ready: being runnable, temporarily stopped (time-out) to let another process execute
- Blocked/Waiting: unable to run until some external event happens, such as I/O completion

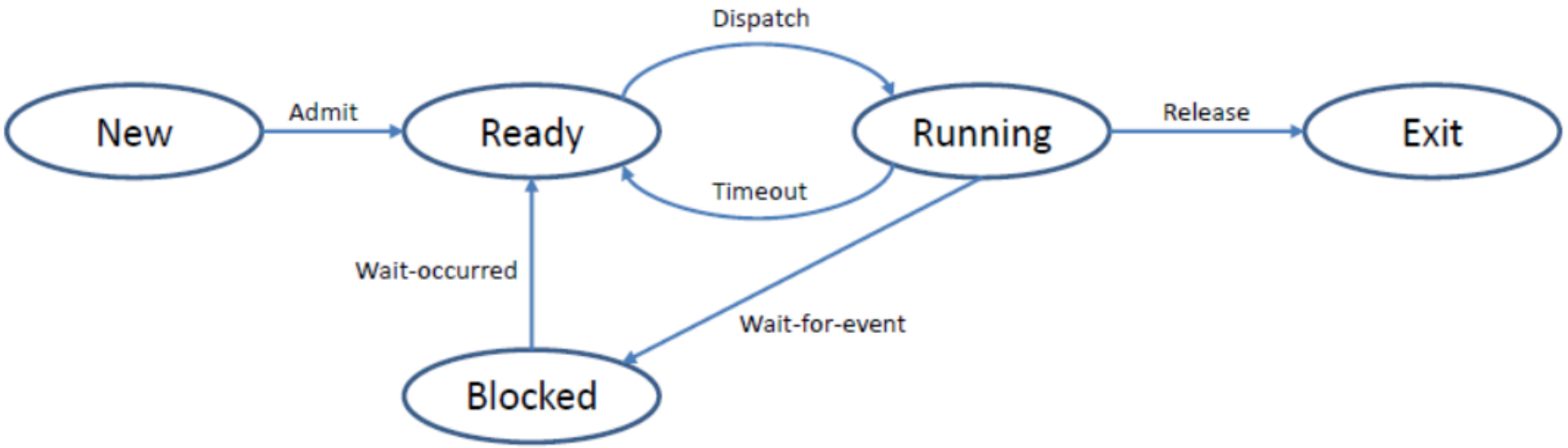
State Transition

- Transition Running – Blocked:
  - Operating systems discovers that process cannot continue right now
- Unix: process is automatically blocked by dispatcher
- Other systems: processes may execute a system call such as “pause” to set themselves into blocked state
- Transition Running – Ready – Running:
  - Performed by dispatcher, without process knowing
  - When the time slice of a process expires, it is set into Ready state
  - Process is transferred back into Running state according to some scheduling policy (performance vs fairness)
- Transition Blocked – Ready
  - Process is transferred into Ready state, when external event occurs

State Transition

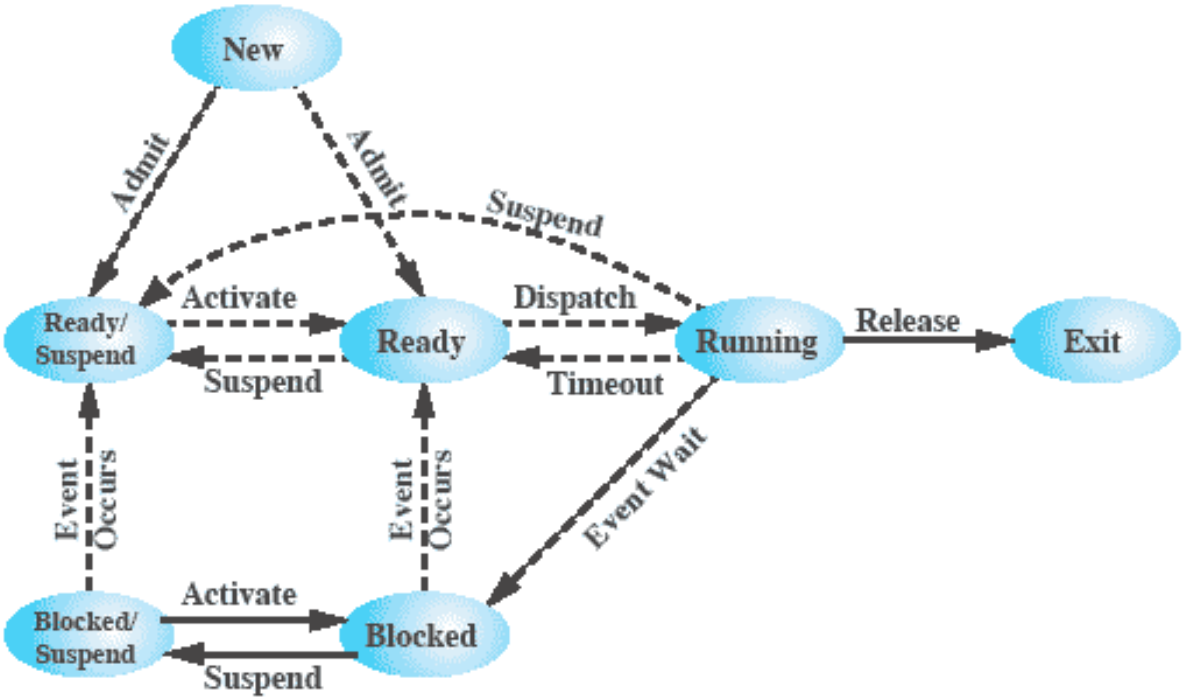
- Scheduling / dispatching a process
  - A process in “Ready” state is chosen and transferred into the “Running” state:
- Program code loaded, the execution continues according to the information held in the PCB
- Suspending a process:
  - transfer from state “Running” into “Ready”
- Process is “preempted”:
  - Process has reached a timeout (maximum allotted time for execution)
  - Process of a higher priority is scheduled
- Process gives up processor voluntarily before timeout (background process)
  - Transfer from state “Running” into “Blocked”
- Process makes a request to operating system in the form of a system service call and has to wait for the call to be completed
  - E.g. request to access a file or shared section of virtual memory
  - E.g. I/O operation that must be completed
  - E.g. Inter-process communication: Process communicates with another process and waits for response

Five-State Process Model



- Process states
  - New: process is created, but not yet admitted to the pool of executable processes by the operating system
- Process Control Block created
- Program may not be loaded yet into main memory
  - Ready: process is prepared for execution and waits to be assigned to a processor
  - Running: process is executed
  - Blocked: a process waits for some event to occur (such as the reception of a signal or the completion of an I/O operation)
  - Exit: process is released from pool of executable processes, because it halted or aborted

Suspended Processes



(b) With Two Suspend States

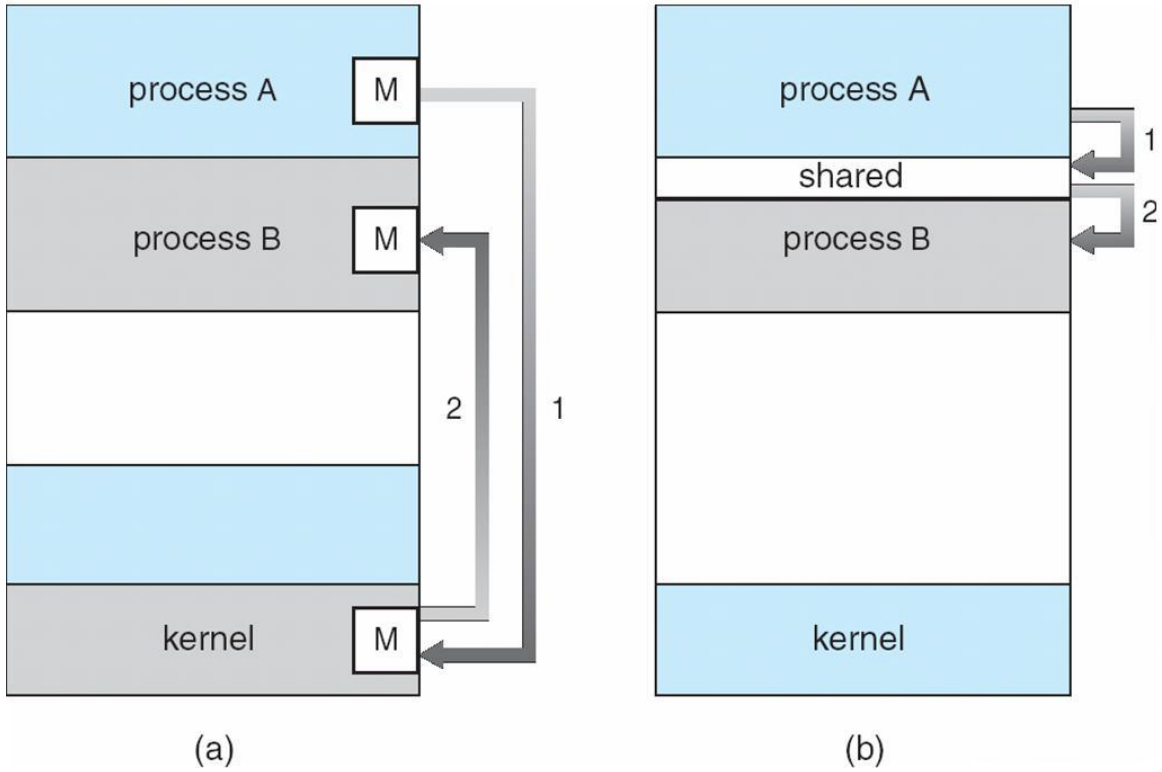
- Processes will be swapped out to disk to free physical memory
- Swapped out processes become suspended
  - I/O operation needed to swap them in
  - Two new states: Blocked-Suspended, Ready-Suspended

Interprocess Communication

- Processes may not operate isolated, but interact and exchange information
- Situations
  - Processes send messages to each other
  - Processes access shared resources
  - Processes coordinate their activities; one process produces information that is consumed by another process
- Processes need an infrastructure to exchange information
  - Information exchange allows processes to cooperate in the performance of tasks
- Two models:
  - Shared Memory
  - Message Passing

Interprocess Communication

- Inter-process communication may be implemented with one of the two methods:



a) Message passing via kernel

b) Using shared memory accessed by both processes

Interprocess Communication

- Reasons for Process cooperation
  - ***Information sharing:*** allow concurrent access to shared information
  - ***Synchronization:*** Computation speedup: decompose your application into sub-elements where each can run as a separate process / thread, can utilize parallel hardware, needs interaction to synchronize activities
  - ***Modularity:*** modular design of software systems, different system functions run as separate processes, needs interaction and information exchange
  - ***Convenience:*** a single user can run multiple applications at the same time, may want to exchange information between them

Shared Memory:

- Two processes may share memory and use it for exchanging information
  - Memory area in a process address space
  - Can be a single shared variable accessible by both processes or complex data structures
- Processes communicate by reading and writing a shared memory space
  - More complex data structures can be allocated in the shared memory area, e.g. a buffer

Message Passing:

- Message passing allows processes to communicate and to synchronize their actions without sharing the same address space
- Important in distributed systems
- IPC facility provides two operations
  - send(message)
  - receive (message)
- Messages can be of fixed or variable size
  - With fixed size, system implementation is straight-forward, but a programmer has to work with fixed sized message formats
  - Variable size has more system management overhead, but is easier to program

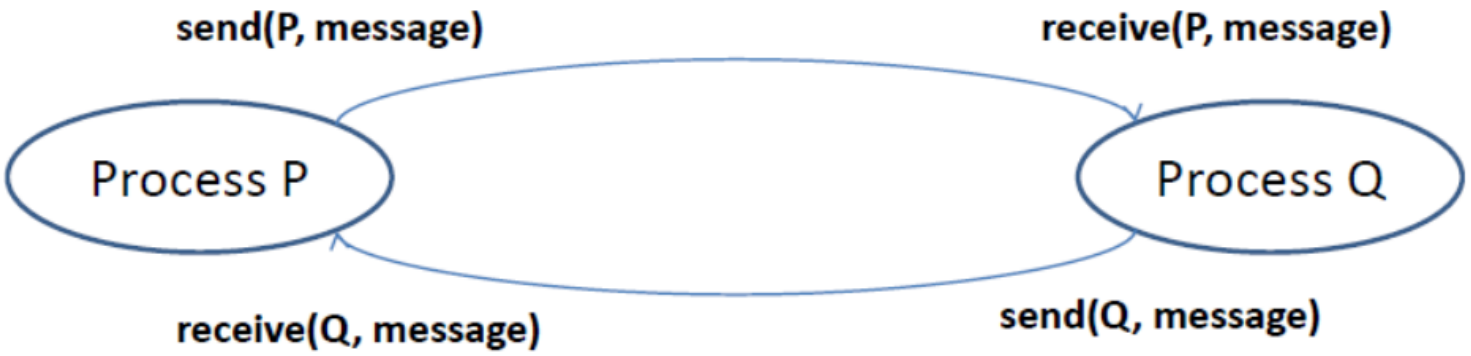
Message Passing:

- Message passing requires a communication link between two processes
  - Physical implementation of a communication link can be based on shared memory or network concepts
- Communication forms:
  - Direct or indirect communication
  - Synchronous or asynchronous communication
  - Automatic or explicit buffering

Message Passing:

Direct Communication:

- Processes must name each other explicitly
  - Send(P, message): Send a message to process P
  - Receive (Q, message): Receive a message from process Q



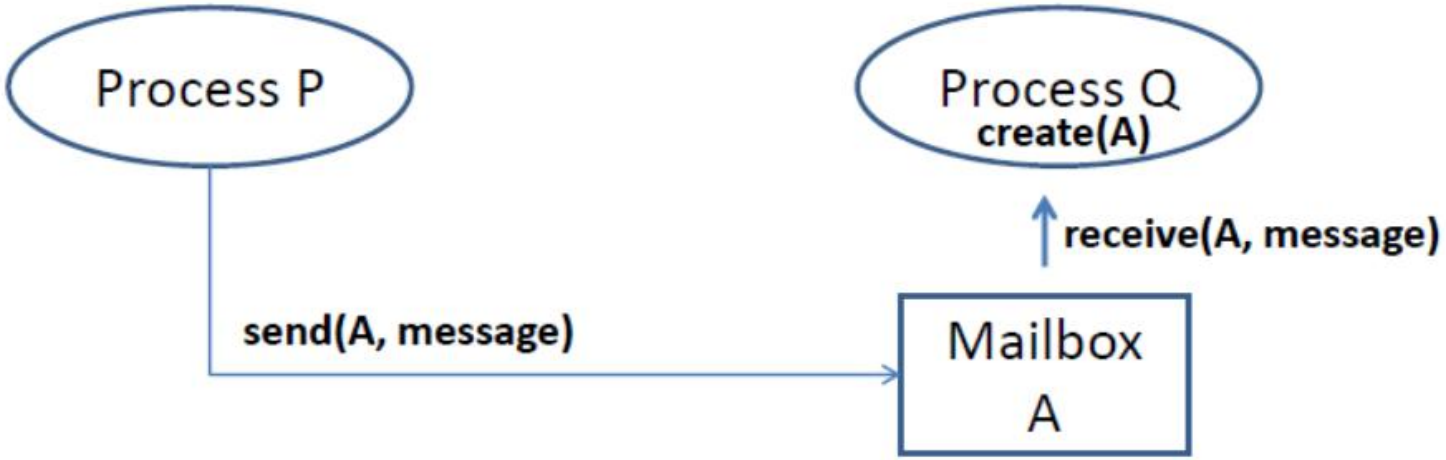
Message Passing:

Indirect Communication:

- Messages are sent to and received via a separate memory area, called mailboxes (also referred as ports)
  - Each mailbox has a unique id
  - Processes can communicate only if they share a mailbox
- Primitives are defined as
  - Send(A, message): Send a message to mailbox A
  - Receive(A, message): Receive a message from mailbox A

Message Passing:

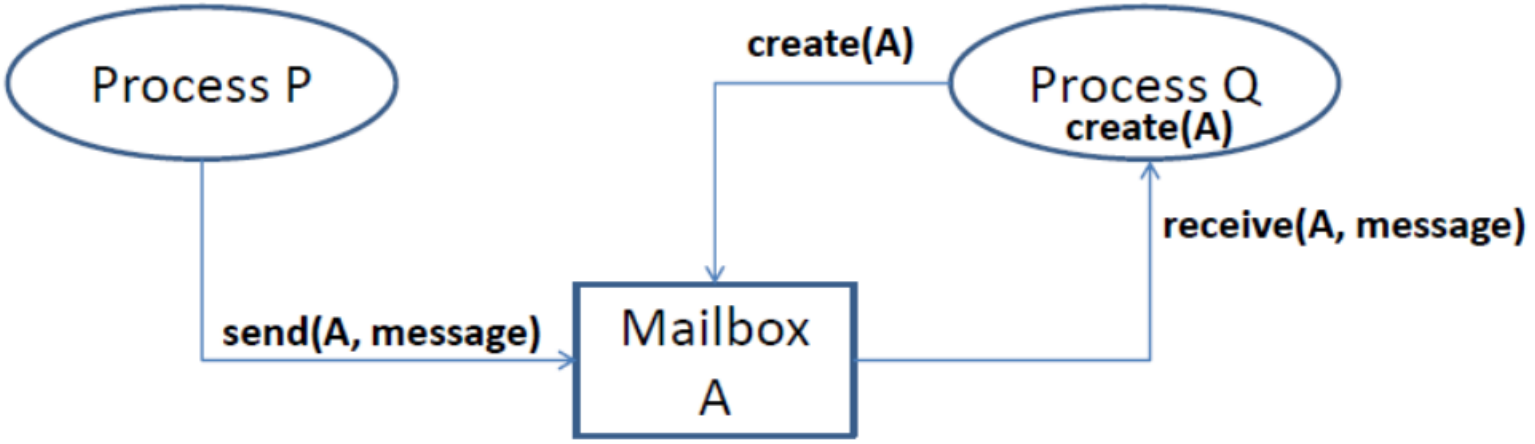
Indirect Communication:



- Mailbox owned by receiving process
  - Receiver regarded as “owner”, can only retrieve / receive messages from mailbox
  - Sender regarded as “user”, can only send / write to mailbox

Message Passing:

Indirect Communication:



- Mailbox owned by Operating System
  - Operating system provides system calls for creating and deleting a mailbox, send and receive messages
  - Creating process becomes the “owner”, can only receive / retrieve messages from mailbox

Process Interaction:

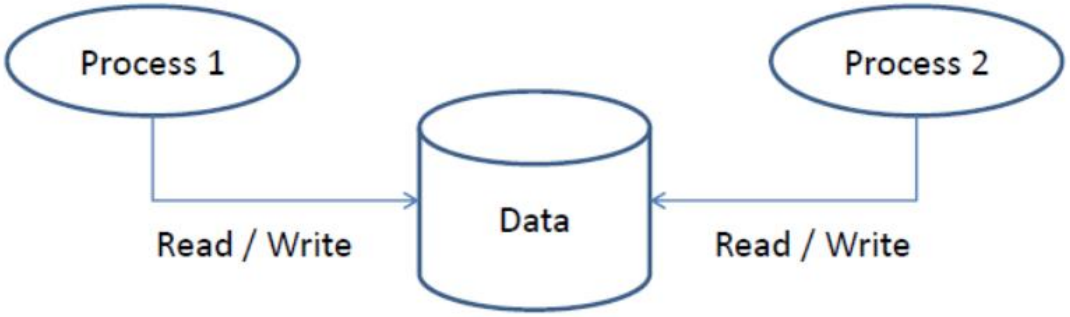
- Processes may interact and exchange information with each other.
- Process interaction creates problems and can lead to inconsistent processing and wrong data manipulations
  - Processes coordinate their activities: one process produces information that is consumed by another process
  - Concurrent access to shared data may result in data inconsistency
- We need mechanisms to ensure the orderly execution of cooperating processes

Concurrency:

- Is a fundamental concept in operating system design
- Processes execute interleaved in time on a single processor
  - Creates the illusion of simultaneous execution
  - Benefits for processing efficiency
- Processes execute truly overlapping / parallel on multiprocessor systems
- Issues
  - Communication among processes
  - Sharing of / competition for resources
  - Synchronization of activities of multiple processes
  - Allocation of processor time to processes

Race Condition:

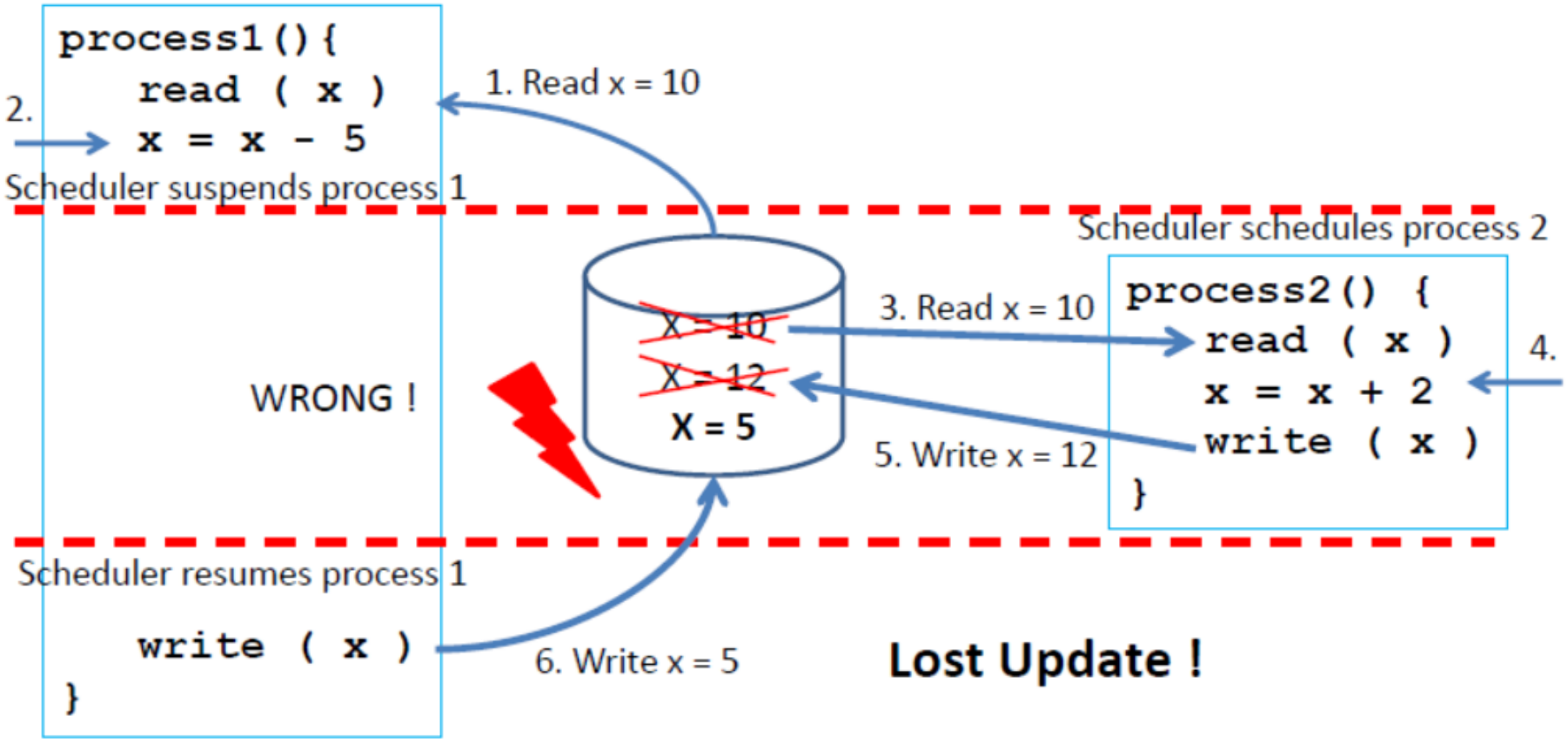
- Occurs when multiple processes / threads read and write shared data items



- The processes race to perform their read / write actions
- The final result depends on the order of execution
  - The “loser” of the race is the process that performs the last update and determines the final value of a shared data item

Race Condition:

- Scenario
  - Process 1 reads shared variable x (x = 10)
  - Process 1 is preempted by scheduler
  - Process 2 continues with its manipulations
  - Process 1 is scheduled and completes its actions



Critical Section:

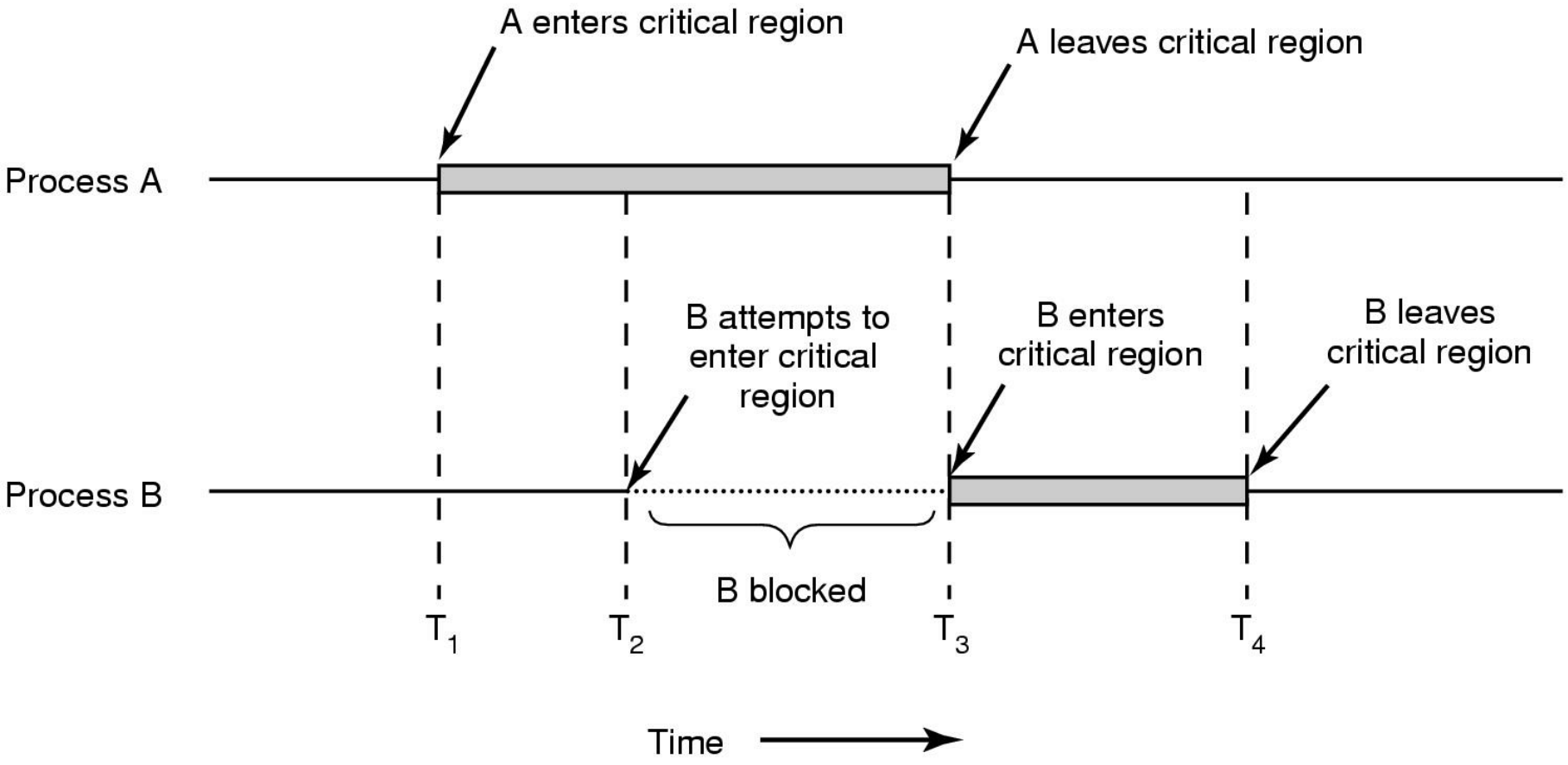
- Critical Section: Part of the program code that accesses shared resource
- In order to guarantee mutual exclusion, we have to control the execution of program code that accesses shared resources
- A race condition can be avoided, if we arrange the execution of processes such that
  - Only one of them is executing its critical section (mutual exclusion)
  - This one process can finish the execution of its critical section, even if it is temporarily suspended
  - Any other process sharing the resource is blocked, in the meantime, from accessing it

Mutual Exclusion:

- It refers to a property of concurrency control where only one task or thread can access a particular resource or perform its critical section at a time, while other threads or tasks are excluded from accessing or modifying the resource until the first thread or task releases it.

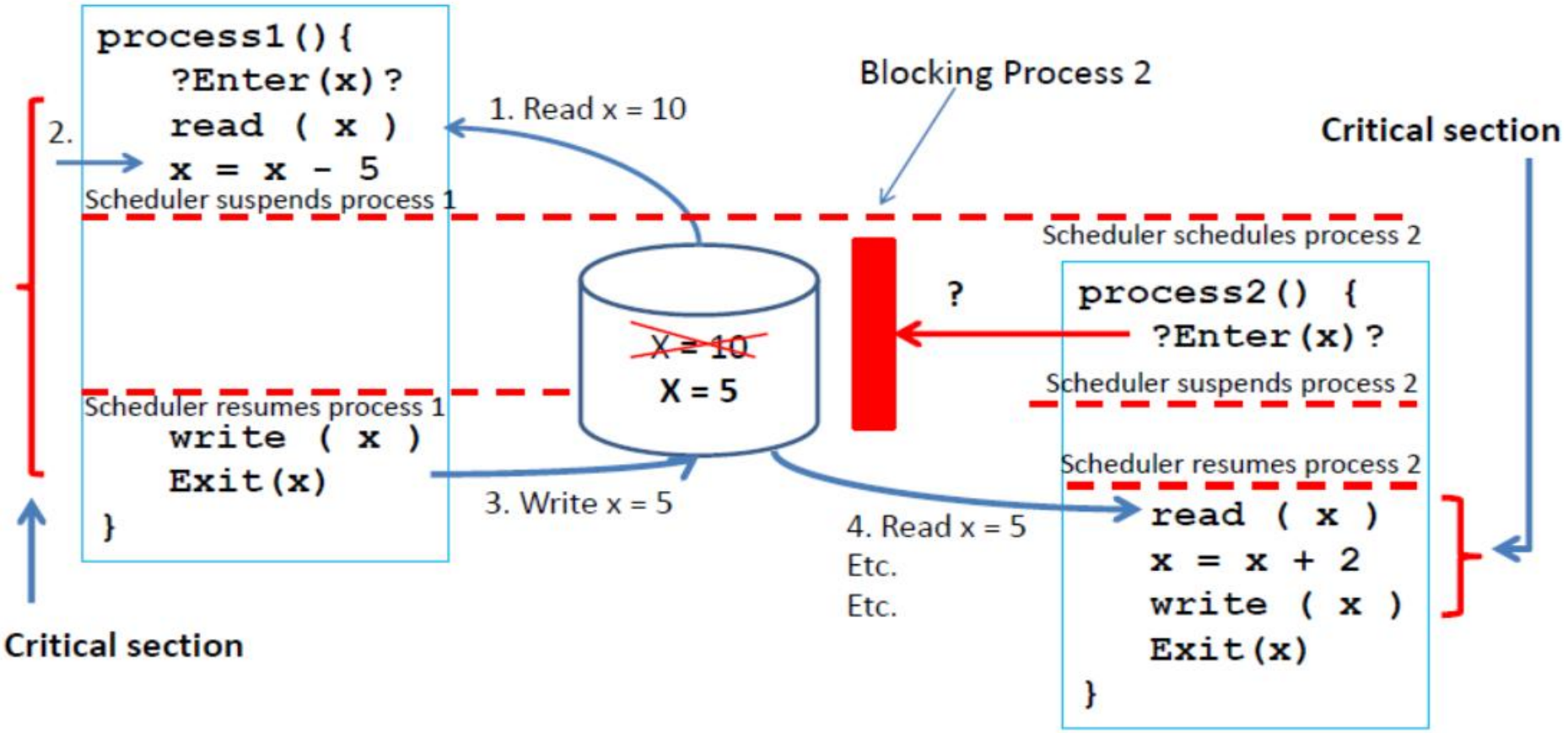
Mutual Exclusion:

- Mutual Exclusion using critical sections



Mutual Exclusion:

- Mutual Exclusion using critical sections
- Each process has a critical section
  - When process 1 entered its critical section, process 2 will encounter a "block" and has to wait
  - Process 1 finishes the execution of its critical section in an "atomic" fashion, even when interrupted by the scheduler



Requirements for Mutual Exclusion:

- No two process may reside at the same time inside the critical section.
- No assumptions are made about relative speeds of processes or number of CPU's.
- No process outside the critical section should block other processes.
- No process should wait arbitrator long to enter it's critical section.

Achieving Mutual Exclusion:

- Mechanism with buzzy waiting for accessing critical region:
  - Disabling Interrupts
  - Lock Variables
  - Strict Alternation
  - Peterson's Solution
  - Test Set Lock (TSL) Instruction
- Mechanism with buzzy waiting for accessing critical region:
  - Sleep and Wakeup
  - Semaphore
  - Monitors
  - Message Passing

Disabling Interrupts

- Interrupt Disabling
  - A process cannot be interrupted until it enables interrupts again
  - Therefore, guarantees mutual exclusion on a uniprocessor system
- Disadvantages
  - Does not work on a multiprocessor architecture
  - The efficiency of execution may degrade as concurrency of processes on one processor is reduced

Lock Variables

- We could introduce a shared variable “lock”
  - Used to indicate whether one of the competing processes has entered critical section
    - If lock == 0, then lock is not set
    - If lock == 1, then lock is set
  - All processes that compete for a shared resource, also share this lock variable
- States
  - A process checks the lock
    - If lock is not set, process sets lock and enters critical section
    - If lock is set, process waits
- Problem
  - As lock variable is a shared resource, race conditions can occur

Strict Alteration

- A turn variable, which can take values 0 or 1, is used to indicate whose turn it is to enter the critical section.
- **Turn value = 0:** When the turn value is set to 0, it means that process P0 will enter into the critical section.
- **Turn value = 1:** When the turn value is set to 1, it means that process P1 will enter into the critical section.

```
Process0:

while(TRUE){
   while ( turn != 0 )
      /* do nothing */ ;
   do_critical_section();
   turn = 1 ;
   do_noncritical_stuff();
}
```

```
Process1:

while(TRUE){
   while ( turn != 1 )
      /* do nothing */ ;
   do_critical_section() ;
   turn = 0 ;
   do_noncritical_stuff();
}
```

Peterson's Algorithm

- Combines strict alteration between two processes with a Boolean flag that indicates that a process is ready or "interested" to enter the critical section
- Shared control variables
  - int turn ;
  - boolean interested[2];
    - interested[0] expresses the interest of process 0
    - interested[1] expresses the interest of process 1

Peterson's Algorithm

```
void enter ( int process )
{
   int i = process ;
   int j = 1 - process;      // we assume process 0 and 1

   interested[i] = TRUE;
   turn = i;
   while ( turn == i && interested[j] == TRUE)
      /* do nothing */ ;
}
```

```
void leave( int process )
{
   interested[process] = FALSE;
}
```

Peterson's Algorithm

- Initially:
  - No process in critical region
    - turn = 0, interested[0] = FALSE, interested[1] = FALSE
- Process 0 calls enter()
  - Sets turn = 0, sets interested[0] =TRUE
  - As interested[1] == FALSE, process enters critical section
- Process 1 calls enter()
  - Sets turn = 1, interested[1] = TRUE,
  - As interested[0] == TRUE, process waits, until process 0 finishes
- Process 0 calls exit()
  - Sets interested[0] = FALSE
- Process 1 enters critical section ...

TSL Instruction

- Implements a “busy wait”: process will do this check until the test indicates a value == 0
  - Process will enter critical section, if test successful
  - Shared memory “lock” is already set to a value > 0
  - When process exits critical section, it will write a value == 0 into

```
process()
{
    while ( testset(LOCK) ) /* do nothing */ ;
    do_critical_section() ;
    LOCK = 0 ;
}
```

TSL Instruction

- The TSL Instruction (Test and Set Lock)
  - Does a Read + Write in one go
  - Does this in one go:
    - Read content of a shared memory location, the variable “lock”, into register and overwrites “lock” with a non-zero value
  - Tests whether a shared variable, used as a “lock”, is set
    - If processor register shows a value == 0, lock is not set
    - If processor register shows a value > 0, lock is set

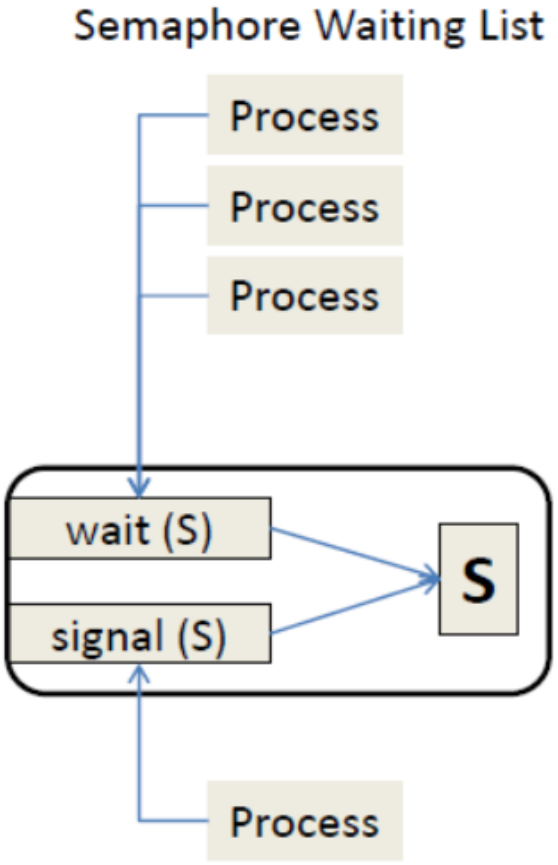
```
enter_section
  TSL REGISTER, LOCK      copy LOCK to register and set lock to 1
  CMP REGISTER, #0        was lock == 0 ?
  JNE enter_section       if it was > 0, LOCK was set, loop again
  RET                     return to caller, critical section free to
                          enter
exit_section
  MOVE LOCK, #0
  RET
```

Sleep and Wakeup

- When a process is not permitted to access its critical section, it uses a system call known as **Sleep**, which causes that process to block.
- The process will not be scheduled to run again, until another process uses the **Wakeup** system call.
- In most cases, **Wakeup** is called by a process when it leaves its critical section if any other processes have blocked.

Semaphores:

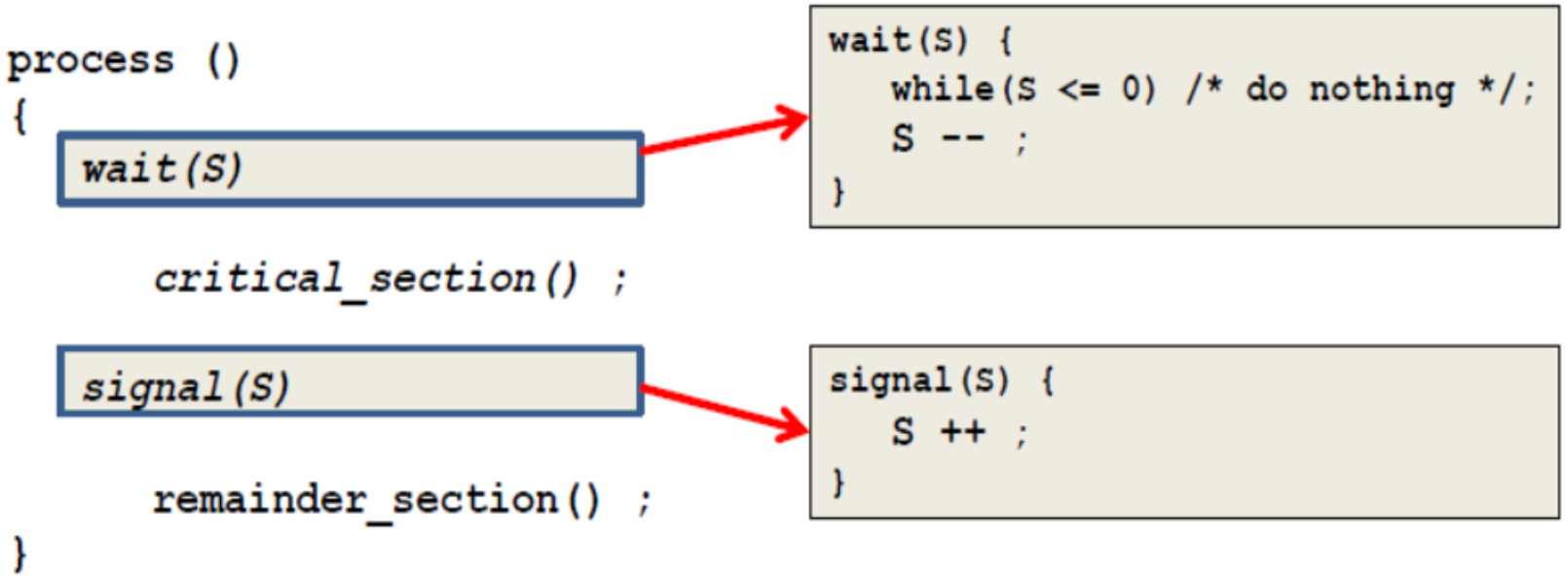
- Synchronization mechanism, no need for Busy Wait
- Two or more processes can communicate by means of simple signals
  - One process can be forced to wait for a signal from other processes
- Semaphore acts as a lock, makes processes wait
- A semaphore is a data structure that contains
  - A counter
  - A waiting queue




```
typedef struct {
   int value ;
   queue plist ;
} semaphore ;
```

Semaphores:

- Semaphore can only be accessed by two atomic functions
  - wait (S) : decrements semaphore counter
    - If S < 0 then processes calling wait(S) are suspended and added to the semaphore waiting queue
  - signal(S) : increments semaphore counter
    - processes calling signal(S) wake up other processes



Semaphores:

- Semaphores are based on a decrement / increment mechanism:
  - The initial value of the semaphore determines how many processes may “pass” (do not have to wait) the semaphore at once
  - If semaphore S >= 0, then process can proceed
  - If semaphore S< 0, then process has to wait for signal

Semaphores:

- Operating systems distinguish between counting and binary semaphores
  - Value of a counting semaphore can range over an unrestricted domain
    - Initial value determines how many processes may pass semaphore in one go
  - Value of a binary semaphore toggles between 0 and 1
- Counting semaphores can be used to control access to a finite set of resources
- Binary semaphores can be used as “mutex locks” to deal with the critical section problem

Counting Semaphore:

- Counting semaphores can be used to control access to a resource with finite capacity
  - Counts how many units of a resource are still available after fulfilling a request
- Example:
  - A room booking system with n rooms, a buffer with n places
- Procedure
  - The semaphore is initialized to the number of units available
  - Each process that wishes to use a resource, calls wait, which decrements the semaphore counter
  - When the count goes to 0, the capacity of a resource is exhausted and any new process will have to wait
  - When a process releases it share of a resource, the counter will be increased

Binary Semaphore:

- Binary semaphores can be used to implement mutex locks
  - N processes share a semaphore “mutex”
  - A mutex is a binary semaphore where the call to both the wait(mutex) and signal(mutex) functions has to be done by the same pro cess

- Value of a binary semaphore toggles between 0 and 1

```
process ()
{
    wait ( mutex ) ;

        critical_section() ;

    signal ( mutex ) ;

        remainder_section() ;
}
```

Semaphore Operation:

```
typedef struct {
   int value ;
   queue plist ;
} semaphore ;
```

```
init(S, val) {
   S.value = val;
   S.plist = empty queue;
}
```

```
wait(semaphore S) {
   S.value-- ;
   if(S.value < 0) {
      add this process to S.plist;
      block();
   }
}
```

```
signal(semaphore S) {
   S.value++ ;
   if(S.value <= 0) {
      remove a process P from S.plist;
      wakeup(P);
   }
}
```

- Initialisation
  - Semaphore must be initialised with a non-negative value
  - The initial value given to the counter controls how many processes are allowed “past” the semaphore at once
- wait() decrements semaphore counter
- signal() increments the semaphore counter

Semaphore Operation:

Semaphore:

```
typedef struct {
   int value ;
   queue plist ;
} semaphore ;
```

```
process ()
{
   wait(S)

      critical_section() ;

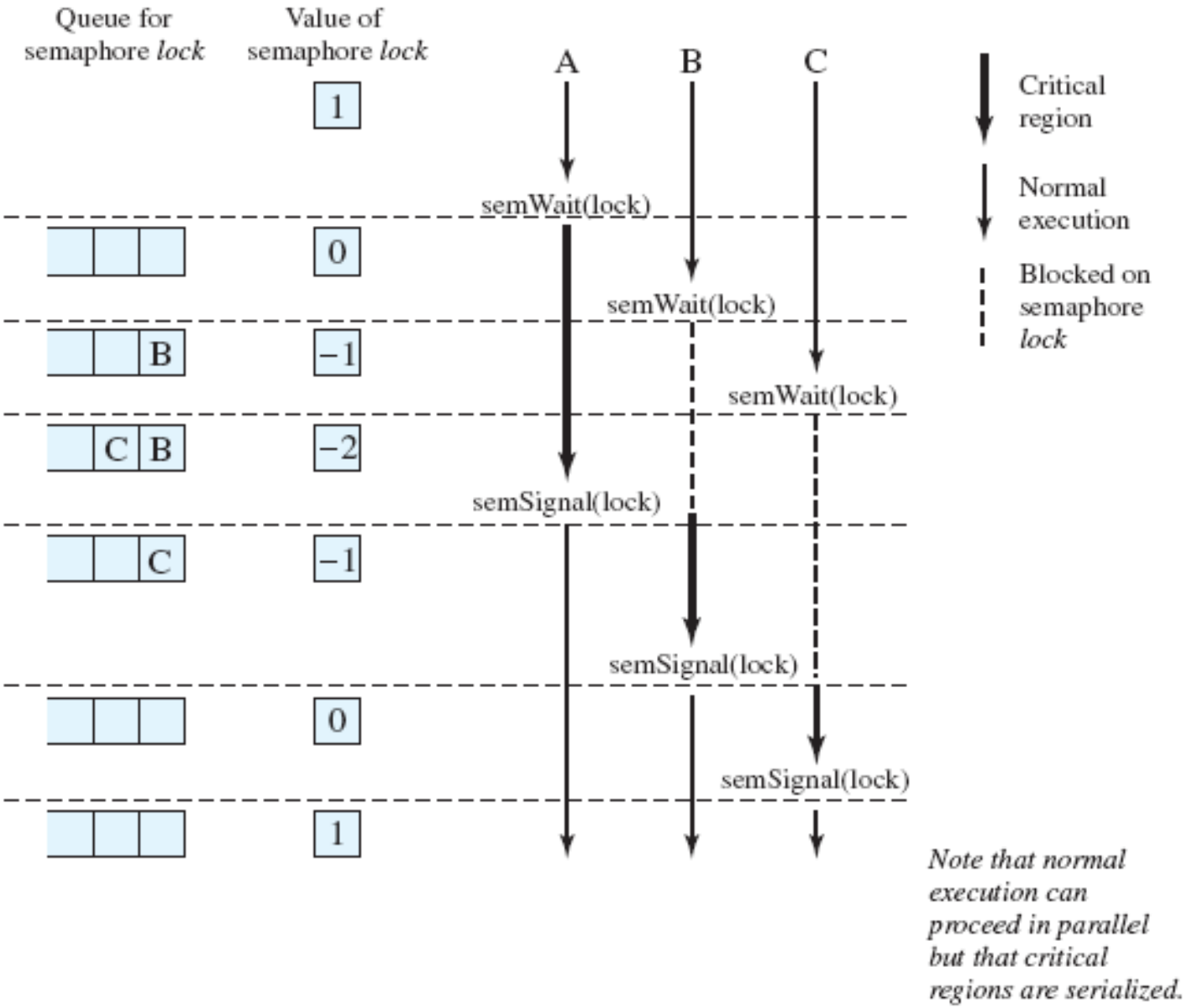
   signal(S)

      remainder_section() ;
}
```

```
wait(semaphore S) {
   S.value-- ;
   if(S.value < 0) {
      add this process to S.plist;
      block();
   }
}
```

```
signal(semaphore S) {
   S.value++ ;
   if(S.value <= 0) {
      remove a process P from S.plist;
      wakeup(P);
   }
}
```

Semaphore Operation:



Binary Semaphore Implementation

Semaphore:

```
typedef struct {
   int value ;
   queue plist ;
} binary_semaphore ;
```

```
process ()
{
   acquire lock

      critical_section() ;

   release lock

      remainder_section() ;
}
```

```
wait(binary_semaphore S) {
   if(S.value == 1) S.value = 0 ;
   else {
      add this process to S.plist;
      block();
   }
}
```

```
signal(binary_semaphore S) {
   if(S.plist is empty) S.value = 1 ;
   else {
      remove a process P from S.plist;
      wakeup(P);
   }
}
```

Strong / Weak Semaphores:

- Strong semaphores
  - Use FIFO queue: the process that has been blocked the longest is released from the queue first
- Weak semaphores
  - The order in which processes are removed from the queue is not specified

Monitor:

- A monitor is a software construct that serves two purposes:
  - enforces mutual exclusion of concurrent access to shared data objects
    - Processes have to acquire a lock to access such a shared resource
  - Support conditional synchronization between processes accessing shared data
    - Multiple processes may use monitor-specific wait()/signal() mechanisms to wait for particular conditions to hold

Monitor Enforce mutual exclusion :

- A monitor consists of
  - One or more procedures
  - Some local data that can only be accessed via these procedures
- When a process calls one of these procedures, it “enters” the monitor
  - The process has to acquire a monitor lock
- The monitor guarantees that
  - Only one process at a time may call one of these procedures and “enter” the monitor
  - All other processes have to wait

monitor pseudocode:

```
monitor example
  buffer b[N] ;
  int in, out ;

  procedure producer()
  . . .
  end;

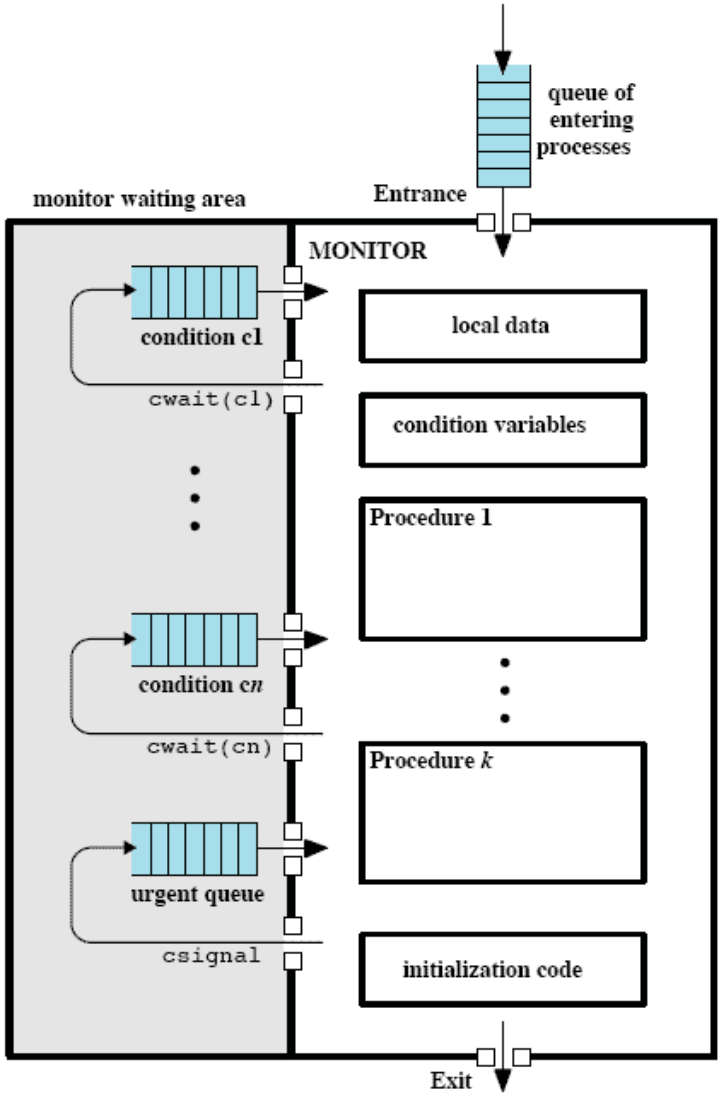
  procedure consumer()
  . . .
  end;
end monitor;
```

Monitor Supporting Process Synchronization:

- A monitor also supports process synchronization with condition variables
  - Only accessible within the monitor
  - Two functions
    - wait(c) : a process calling this functions is suspended, releases monitor lock, waits until a signal based on condition c is received, reacquires lock
    - signal(c) : resume one of the processes waiting
- A monitor may maintain a set of these condition variables
- Processes may wait for any of these condition variables
  - A suspended process releases the monitor lock
- For each condition variable, the monitor maintains a waiting queue

Monitor Operations :

- Process “enters” a monitor
  - It calls one of the procedures
    - Process acquires lock, may have to wait
- One process currently in the monitor
  - Process may call wait(condition)
    - made to wait for a condition in a condition queue
  - Process may call signal(condition)
    - This resumes one of the processes waiting for this conditional signal



Message Passing:

- Sender and receiver process will use at least the following operations:
  - send (destination, message)
  - receive (source, message)
- When processes interact with one another, two fundamental requirements must be satisfied:

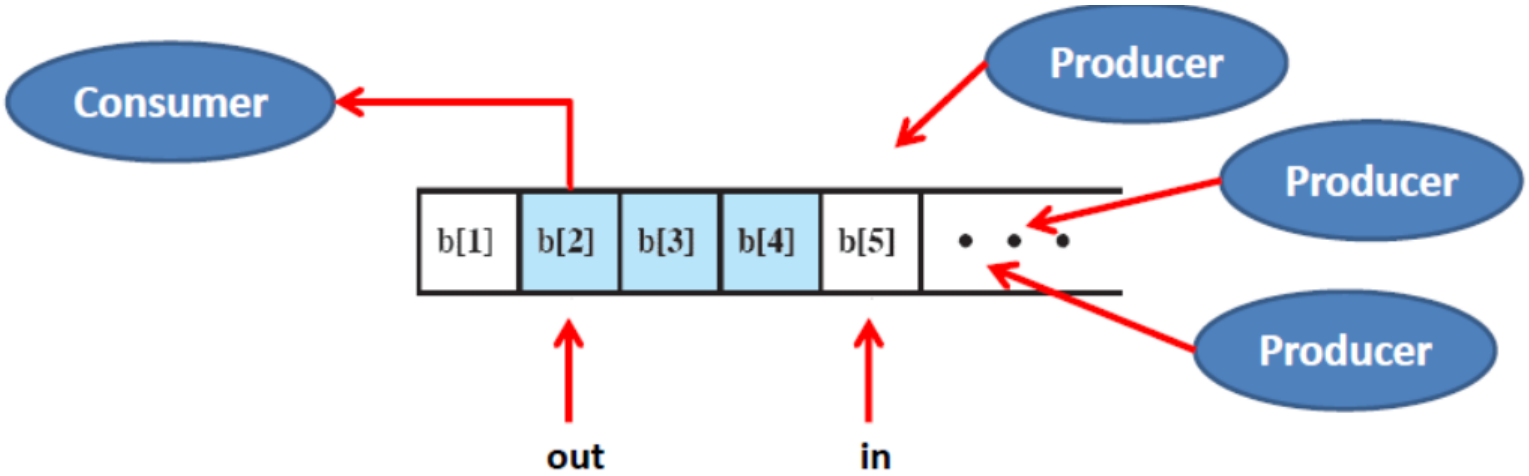
synchronization	communication
• to enforce mutual exclusion	• to exchange information

Classic Problems of Synchronization:

- Producer-Consumer (bounded-buffer) problem
- Reader-Writer Problem
- Dining Philosophers Problem
- The Sleeping Barber Problem

Producer – Consumer Problem:

- Producer and consumer processes exchange data items via a buffer
- One or more producers put data into the buffer
- One consumer takes information out of the buffer
- ***Objective:*** prevent any overlap of buffer operations !



Producer – Consumer Problem:

General Situation:

- one or more producers are generating data and placing these in a buffer
- a single consumer is taking items out of the buffer one at time
- only one producer or consumer may access the buffer at any one time

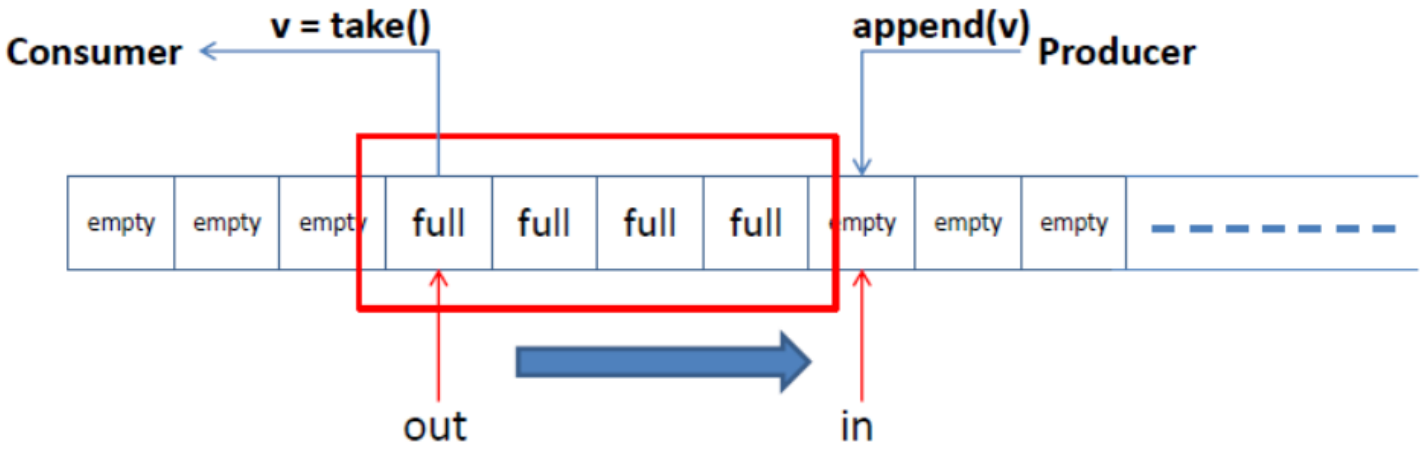


The Problem:

- ensure that the producer can't add data into full buffer and consumer can't remove data from an empty buffer!

Producer – Consumer Problem:

- Assumption: infinite Buffer
- Producer can append elements to buffer any time (because no buffer restrictions)
- Consumer has to wait for data
- Two buffer pointers
  - ***"in"*** : points to next free place in buffer
  - ***"out":*** points to next data element in buffer that can be read



Producer – Consumer Problem:

```
Initialisation:
in = out = 0 ;
```

```
Producer:
while ( true )
{
   v = produce_item() ;

   b[in] = v ;
   in ++ ;
}
```

```
Consumer:
while ( true )
{
   while ( in <= out )
      /* do nothing */ ;
   v = b[out] ;
   out ++ ;
   consume_item ( v ) ;
}
```

- Consumer checks whether there is something in the buffer
- How to implement mutual exclusion between producer and consumer ?

Producer – Consumer Problem:

- Mutual exclusion
  - Only one process may access buffer at any time
  - Safeguarded with semaphore:
    - We use a semaphore “mutex” to control mutual exclusion
- Condition for consumer:
  - Consumer can only read/remove elements from buffer, if there is at least one unread data element stored in buffer
  - Semaphore
    - We use a counting semaphore “N” that counts the number of data elements stored in the buffer

Producer – Consumer Problem:

```
Initialisation:
in = out = 0 ;
init_semaphore(mutex,1); init_semaphore(N,0);
```

```
append(v){
    b[in] = v;
    in++ ;
}

take(){
    v = b[out];
    out++;
    return v;
}
```

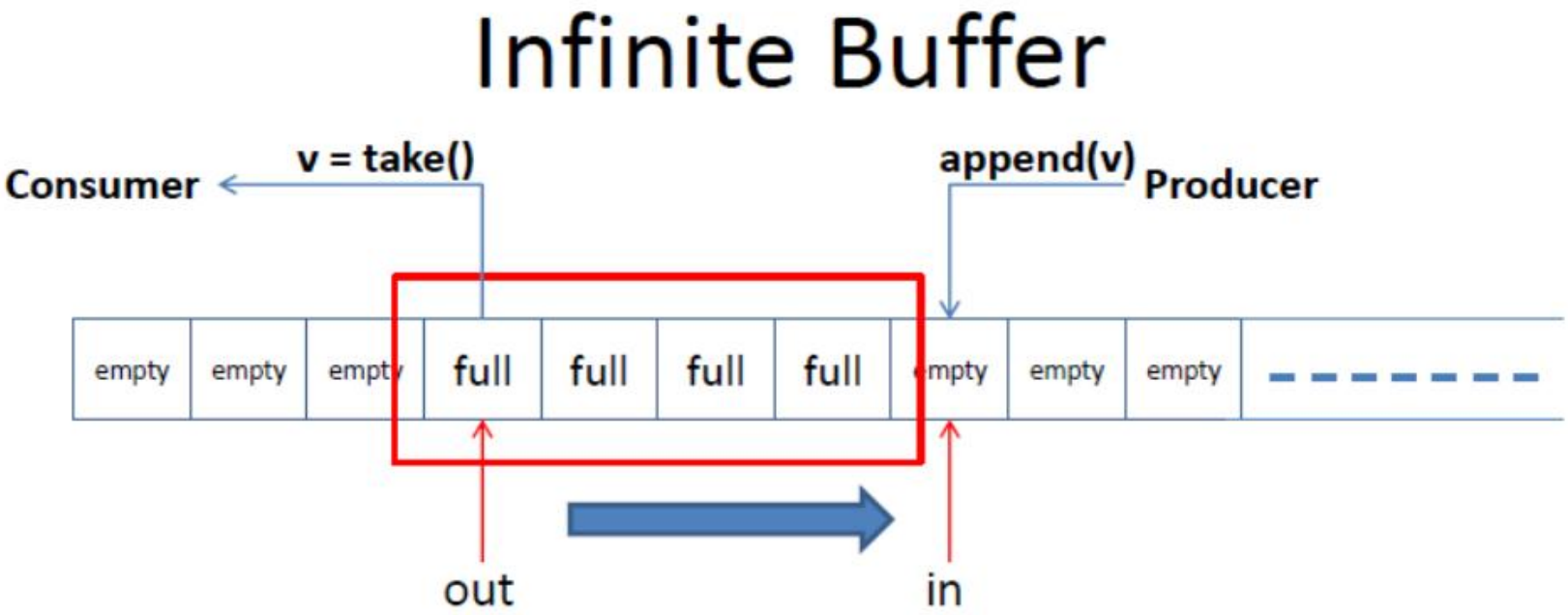
```
Producer:
while ( true )
{
    v = produce();
    wait(mutex);
    append(v);
    signal(mutex);
    signal(N);
}
```

```
Consumer:
while ( true )
{
    wait(N);
    wait(mutex):
    v = take() ;
    signal(mutex);
    consume(v) ;
}
```



- Two semaphores
  - Saveguard update on buffer (mutex)
  - Count elements currently stored in buffer (N)

Producer – Consumer Problem:

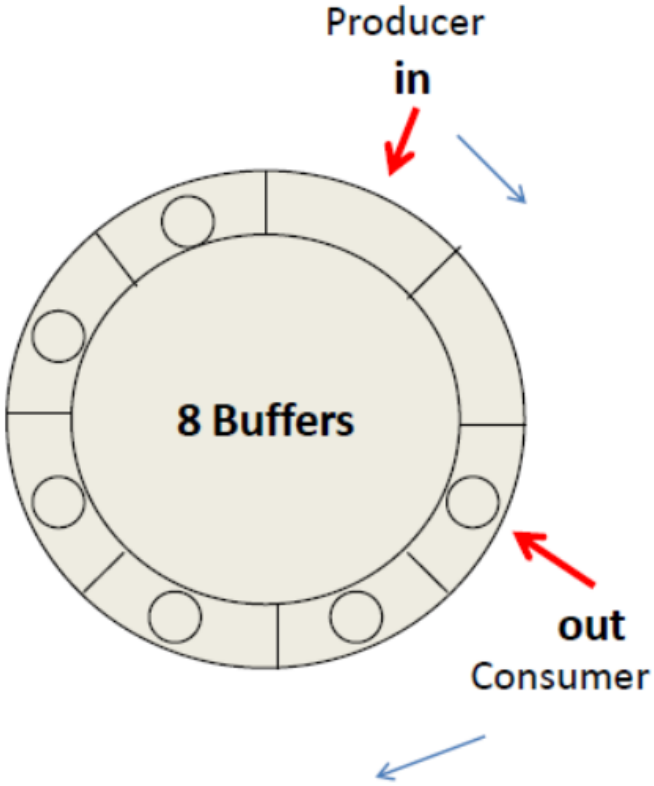


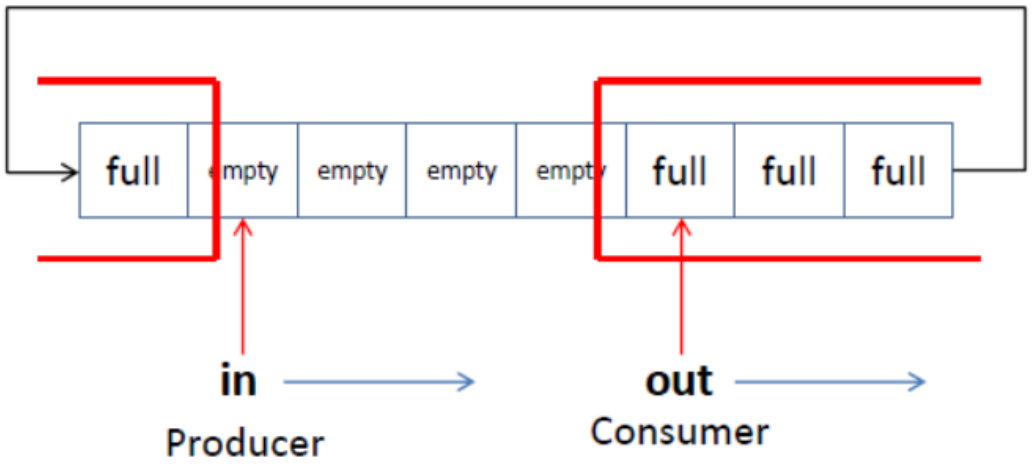
- If we assume an infinite buffer, a "window of full places" appears to move across the buffer
  - Producer always takes a new empty place
  - Places that become free again, are never reused
- Infinite buffer can be implemented with a bounded buffer or "ring buffer"

Producer – Consumer Problem:

Bounded Buffer

- Implemented as a Ring Buffer
  - If “in” pointer reaches last location in buffer, it is reset to first location
  - If “out” pointer reaches last location in buffer, it is reset to first location





Producer – Consumer Problem:

Bounded Buffer

```
Initialisation:
in = out = 0 ; n = size(buffer) ;
```

```
Producer:
while ( true )
{
    v = produce_item() ;

    b[in] = v ;
    in = (in+1) mod n;
}
```

```
Consumer:
while ( true )
{
    while ( in <= out )
        /* do nothing */ ;
    v = b[out] ;
    out = (out+1) mod n ;
    consume_item ( v ) ;
}
```

- The buffer pointers are incremented modulo the buffer size

Reader-Writer Problem:

- Processes that share a resource, e.g. a database, may perform read and write operations
- Write operations are critical
  - As it is a change to the shared data object, only one writer at a time may access the data object
  - All other processes, readers and writers, must be excluded from access
- Read operations are not critical
  - Many readers at the same time may read a shared data object
- Writers must have exclusive access to shared data
- Readers can access shared data simultaneously

Reader-Writer Problem:

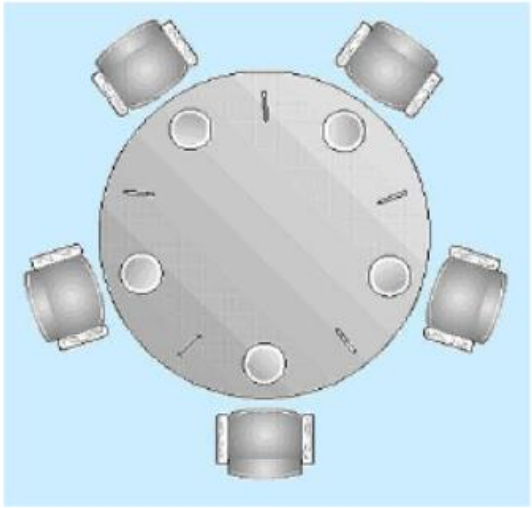
- “First Reader-Writer Problem”
  - Readers have priority
  - No reader is kept waiting, unless a writer has already obtained permission to access the shared object
  - Explanation:
    - First reader gets lock on shared object via semaphore, calling wait()
    - Subsequent readers are simply admitted, without checking the semaphore, using a reader counter
    - The last reader will free the lock by calling signal() on the semaphore

Reader-Writer Problem:

- “Second Reader-Writer Problem”
  - Writers have priority
  - Once a writer is ready, it performs its write operation as soon as possible
  - Explanation: if a writer is waiting to access a shared object, no new reader may start reading

Dining Philosophers Problem:

- Five philosophers sit around a table for dinner
- There are only 5 forks on the table, two neighboring philosophers share one fork



```
Philosopher:
while(TRUE) {
  Think();
  Grab first fork;
  Grab second fork;
  Eat();
  Put down first fork;
  Put down second fork;
}
```

- Each philosopher needs two forks to eat
  - How many of them can eat at the same time?

Dining Philosophers Problem:

- First attempt:
  - Each philosopher represented as a process
  - Each fork represented as a semaphore
  - Each semaphore initialized to 1
- Leads to deadlock, if all philosophers grab their left fork simultaneously
  - Will wait forever for a right fork to become available

```
Initialisation:
   init(fork[1..5],1);
```

```
Philosopher i:
void philosopher( int i ) {
   while(TRUE) {
      think() ;
      wait(fork[i]);
      wait(fork[i+1] mod 5);
      eat();
      signal(fork[i+1] mod 5);
      signal(fork[i]);
   }
}
```

Dining Philosophers Problem:

- Problem occurs if all philosophers want to eat at the same time
- We can introduce timeout:
  - All philosophers pick up their left fork simultaneously
  - They see that right fork is not available and put left fork down again
  - They wait for a set time, pick up left fork again simultaneously
- Situation of starvation: they will never eat

Dining Philosophers Problem:

- Practical Solution
  - Each philosopher waits a random time, before trying again to acquire forks
  - E.g.: Ethernet protocol: if two computers want to send a packet at the same time – collision, both computers wait a random time to try again, hopefully no collision next time
  - Problem: although random time delay, we cannot guarantee that there is no collision next time

Dining Philosophers Problem:

- 5 philosophers try to eat, allow only 4 philosophers at the table
- at least one philosopher has access to two forks at a time,
- extra semaphore “table” set to 4
- No deadlock, no starvation

```
Initialisation:
   init(table,4);
   init(fork[1..5],1);
```

```
Philosopher i:
void philosopher( int i ) {
   while(TRUE) {
      think() ;
      wait(table);
      wait(fork[i]);
      wait(fork[i+1] mod 5);
      eat();
      signal(fork[i+1] mod 5);
      signal(fork[i]);
      signal(table);
   }
}
```

Dining Philosophers Problem:

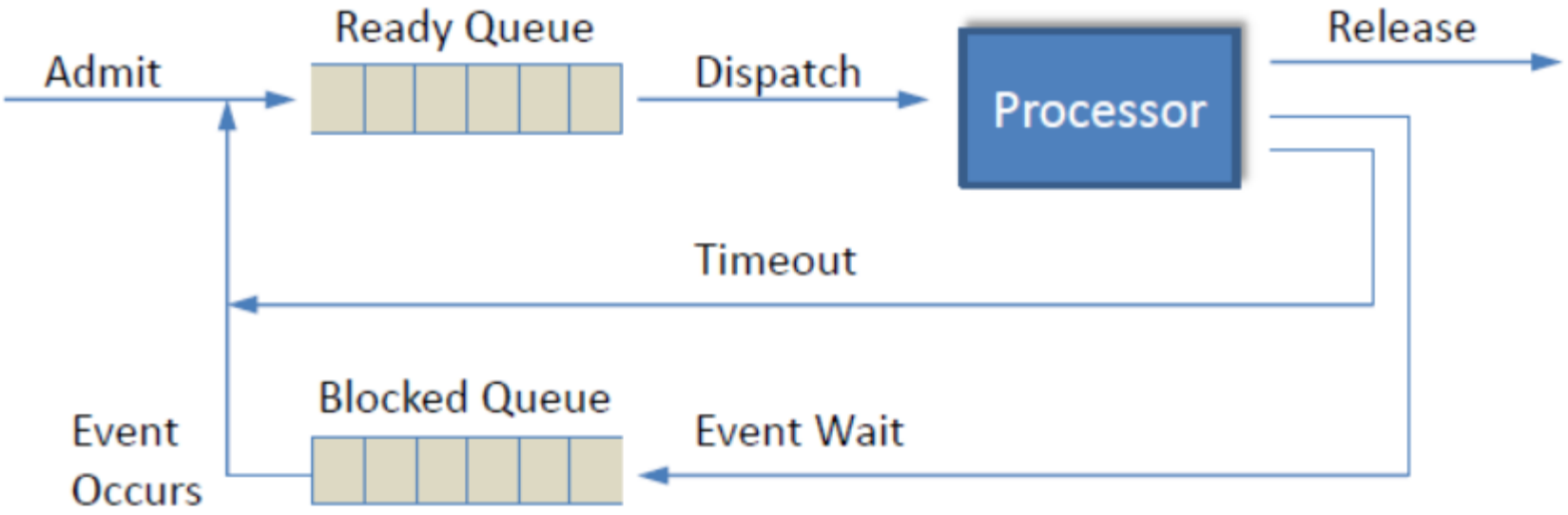
- Other possible remedies
  - Goal: avoid deadlock / starvation, at least one philosopher should be able to eat
  - Allow a philosopher to pick up the two forks only if both are available at the same time
    - We need extra critical section for this
  - Use an asymmetric solution
    - Odd philosophers pick first the left and then the right fork
    - Even philosophers pick first the right and then the left fork

The Sleeping Barber Problem:

- Self Study!

Process Scheduling

- Process Scheduling
  - Decides which process to execute next
  - is the activity of selecting the next process in the Ready queue for execution on a processor



Process Scheduling

- Scheduling can be preemptive or non-preemptive
- Non-preemptive:
  - once a process is scheduled, it continues to execute on the CPU, until
    - it is finished (terminates)
    - It releases the CPU voluntarily (cooperative scheduling behavior, Ready queue)
    - It blocks due to I/O interrupts or because it waits for another process
    - No scheduling decision is made during clock interrupts, process is resumed after this interruption (unless a process with higher priority becomes ready)
- Preemptive:
  - A scheduled process executes, until its time slice is used up
  - Clock interrupt returns control of CPU back to scheduler at end of time slice
    - Current process is suspended and placed in the Ready queue
    - New process is selected from Ready queue and executed on CPU
  - Used by most operating systems

Scheduling Criteria

	User-Oriented	System-Oriented
Performance	Response Time Turnaround Time Waiting Time	Throughput Processor Utilisation
Other	Predictability	Fairness Priorities Resource Balance

Scheduling Performance Criteria

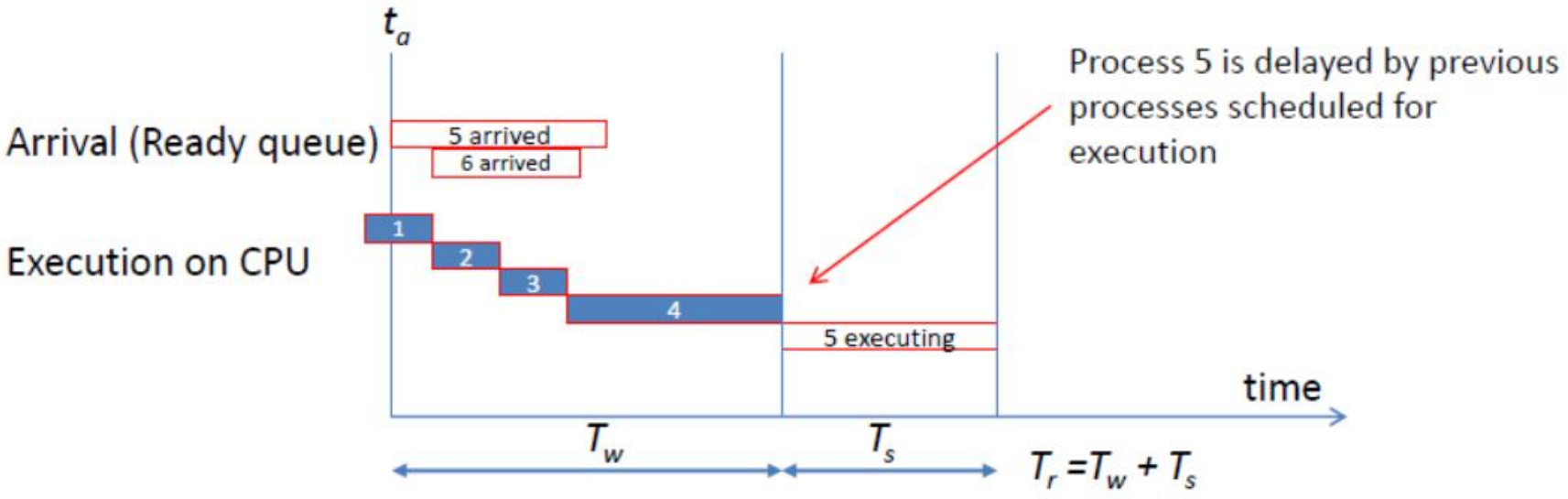
- CPU utilization
  - Keep the CPU as busy as possible
- Throughput
  - Number of processes that complete their execution per time unit
- Response time
  - The time it takes from when a request was submitted until the first response is produced by the operating system (latency)
- Turnaround time
  - Total amount of time to execute one process to its completion
- Waiting time
  - Total amount of time a process has been waiting in the Ready queue

Optimization Criteria

- Maximize CPU utilization
- Maximize throughput
- Minimize turnaround time
- Minimize waiting time
- Minimize response time

Optimization Criteria

- ***Arrival time ts:*** time when process became “ready” (again)
- ***Wait time Tw:*** time spent waiting for CPU
- ***Service time Ts:*** time spent executing on the CPU
- ***Turnaround time Tr:*** total time spent waiting + executing



Scheduling Algorithms

- First Come First Serve scheduling (FIFO)
- Shortest Job first scheduling
- Shortest Remaining Time First
- Priority Scheduling
- Round-Robin Scheduling
- Real-Time Scheduling
- Lottery Scheduling
- Fair-Share Scheduling

First Come First Serve

- Example:
  - Three processes arrive at the same time (t = 0)
  - They have different CPU burst times
- We assume the arrival order **P1, P2, P3**

Process	Burst Time
P1	24
P2	3
P3	3



- Waiting times
  - P1 has 0 time units waiting time
  - P2 has 24 time units waiting time
  - P3 has 27 time units waiting time
  - Average waiting time: (0 + 24 + 27)/3 = 17

First Come First Serve

- We may get a better responsiveness with a different arrival order
- We assume the arrival order: **P2, P3, P1**
- The schedule of these three processes:



- Waiting times : P2 = 0, P3 = 3, P1 = 6
- Average waiting time: (0 + 3 + 6)/3 = 3

First Come First Serve

- Favors long processes
- CPU-intensive processes (long burst times)
- Disadvantages
  - Leads to poor utilization of CPU and I/O devices
  - Average waiting time is highly variable
    - Short jobs may wait behind large ones

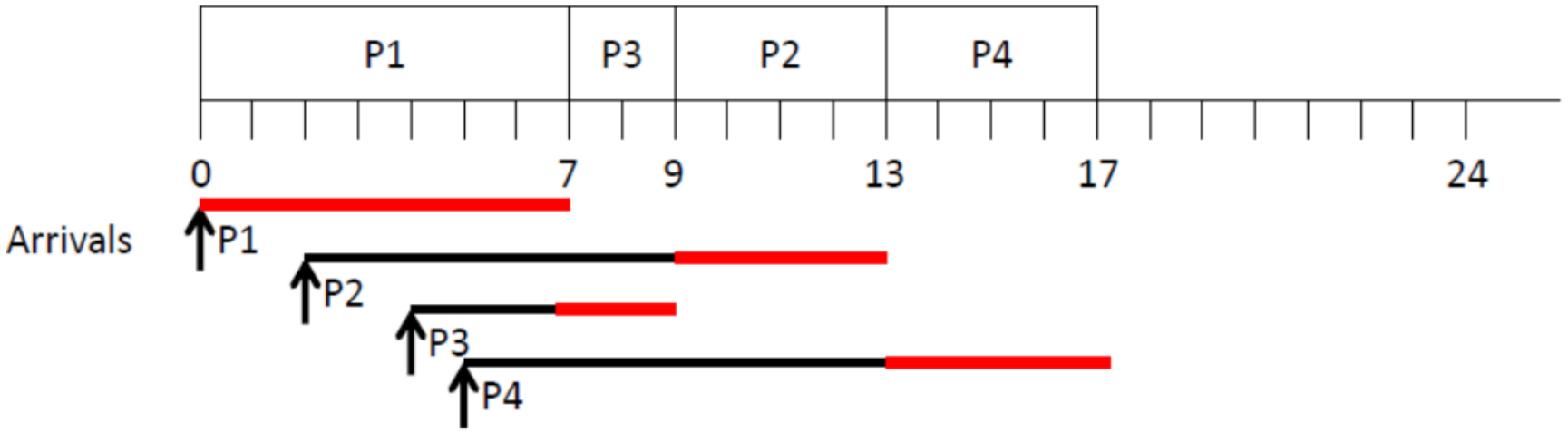
Shortest Job First(SJF)

- Non-preemptive scheduling policy that tries to schedule the “shortest” process first
  - Process with shortest expected processing time
- Difficulty
  - Necessity to estimate the required CPU processing time of each process
    - For batch jobs: past experience, specified manually
    - For interactive jobs: analyze bursts of CPU time
- Short processes will jump to the head of the queue
  - Possibility of starvation for processes with longer execution time

Shortest Job First(SJF)

Process	Arrival Time	Total CPU Time
P1	0	7
P2	2	4
P3	4	2
P4	5	4

- SPN Schedule



- Average Waiting Time: (0 + 7 + 3 + 8)/4 = 4.5

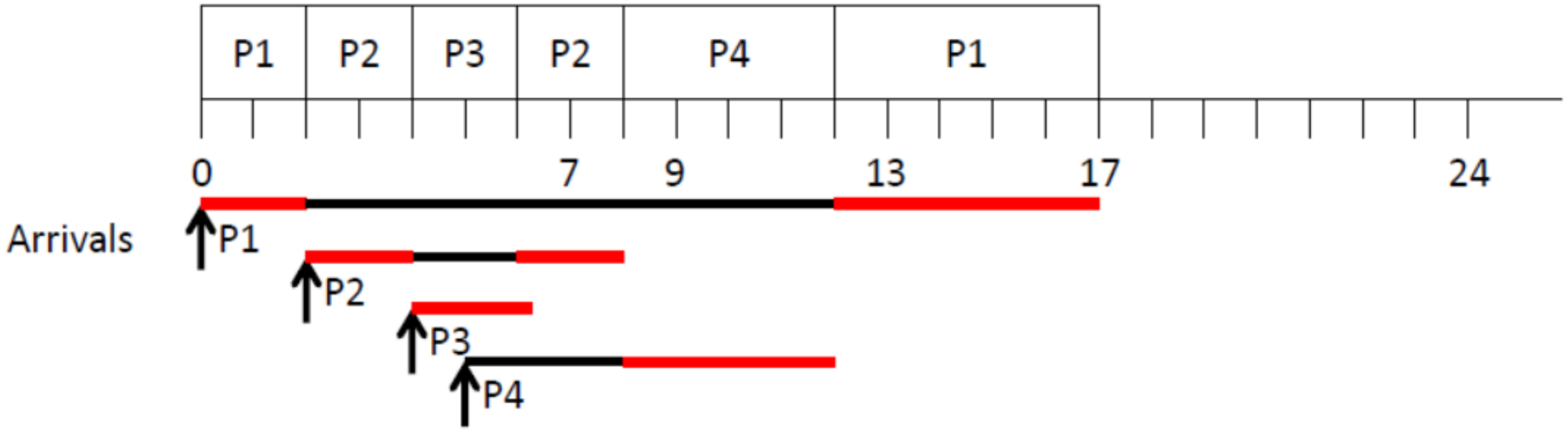
Shortest Remaining Time First (SRTF)

- Preemptive version of the SPN scheduling policy
  - Scheduler tries to select the process whose remaining run time is the shortest
- Same selection function as SPN
- Processes currently executing on CPU are preempted if a job with shorter estimated CPU time becomes ready
- Better turnaround performance
  - Shorter job is given immediate preference to a longer running job
  - Reduces waiting times

Shortest Remaining Time First (SRTF)

Process	Arrival Time	Total CPU Time
P1	0	7
P2	2	4
P3	4	2
P4	5	4

- SRT Schedule



- Average Waiting Time: (10 + 2 + 0 + 3)/4 = 3.75

Priority Scheduling

- A priority number (integer) is associated with each process
  - Smallest integer = highest priority
- The CPU is allocated to the process with the highest priority
  - Preemptive / non-preemptive form of priority scheduling
- SPN is a priority scheduling algorithm where priority is the predicted next CPU burst time
- Starvation
  - Shows a problem of SPN – Starvation
  - Low-priority processes may never execute
- Solution to Starvation
  - Introduce concept of “Aging”: as time progresses, increase the priority of the process
    - Priority = estimated CPU time – waited time so far

Round Robin (RR, Time Slicing)

- Is a preemptive scheduling policy
- Time quantum
  - Each process gets a fixed unit of CPU (usually 10-100 ms)
  - After time quantum has elapsed, the process is preempted and added to the end of the Ready queue
    - Processes scheduled in a cyclical fashion (always same sequence of processes) – round robin
- Fairness: Each process gets an equal time quantum
  - If the time quantum is q time units and if there are n processes in the Ready queue, each process get 1/n of the CPU time, in chunks with a size of at most q time units
  - No process waits more than (n-1)q time units

Round Robin (RR, Time Slicing)

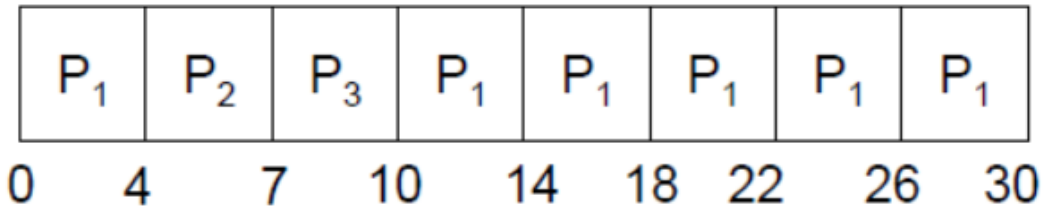
- Performance
  - If q is large: round-robin becomes FCFS
  - If q is small: fair, starvation-free, high responsiveness
    - We have to be careful: q still has to be considerably larger than the time needed for a context switch, otherwise the dispatcher consumes more CPU time than the actual processes it dispatches
- Disadvantage: I/O intensive processes are overtaken by CPU-intensive processes
  - I/O intensive processes block early for I/O, are not using their full time quantum, will be overtaken by CPU intensive processes

Round Robin (RR, Time Slicing)

- Example, time quantum q = 4

Process	CPU Time
P1	24
P2	3
P3	3

- Schedule



- Typically, higher average turnaround than SJF, but better response

Real-Time Scheduling

- In real-time scheduling, time is essential
  - A process has to meet deadlines reliably
- A set of processes has to be scheduled so that all of them meet deadlines reliably
- In this, the process that require immediate execution are managed, often with strict timing constraints.
- It is crucial for systems that need to respond quickly, such as control systems and embedded systems.

Lottery Scheduling

- Basic idea:
  - Give processes “lottery tickets” for various resources, such as CPU time
  - For each scheduling decision, a lottery ticket is chosen at random and the process holding that ticket gets resource
  - E.g.: CPU scheduling
    - Scheduler holds lottery 50 times per second
    - Each winner receives 20msec of CPU time

Lottery Scheduling

- Manipulations
  - Prioritization
  - Ticket exchange
- Prioritizing Processes
  - More important processes can be given extra tickets, increases their odds of winning
  - E.g.: 100 tickets, one process holds 20 of them, therefore a 20% chance of winning, in the long run will receive 20% of CPU time
- Ticket exchange
  - A client process may give tickets to its server to increase the server's chances of getting CPU time earlier

Fair-Share Scheduling

- Fair-share scheduling takes ownership into account
  - Recognizes the fact that processes are owned by users
  - Each user get's fair share of resources (and not the single process), independent of the number of processes run
- Applications, run by a user, may consist of a set of processes
  - User experience (responsiveness) depends on the applications overall performance
- Can be extended to user groups

Threads

- A **thread** is the smallest unit of execution within a process.
- Threads are often referred to as **lightweight processes** because they share the same resources (e.g., memory, code, and files) of the parent process but operate independently.
- Threads enable **concurrent execution**, improving system performance and responsiveness.
- Each thread has its own **program counter**, **register set**, and **stack space**, but threads within the same process share the **code section**, **data section**, and **OS resources** like open files and signals.

Why do We Need Threads?

- Since threads use the same data and code, the operational cost between threads is low.
- Creating and terminating a thread is faster compared to creating or terminating a process.
- Context switching is faster in threads compared to processes.
- It takes far less time to create a new thread in an existing process than to create a new process.
- Threads can share the common data, they do not need to use Inter- Process communication.
- Context switching is faster when working with threads.
- It takes less time to terminate a thread than a process.

Multithreading

- Multithreading is the ability of an operating system to support multiple threads of execution within a single process
- Threads are described by the following:
  - Thread execution state
    - running, ready, blocked
  - Thread Control Block
    - A saved thread context when not running (each thread has a separate program counter)
  - An execution stack
  - Some per-thread static storage for local variables
  - Access to memory and resources of its process, shared with all other threads of that process

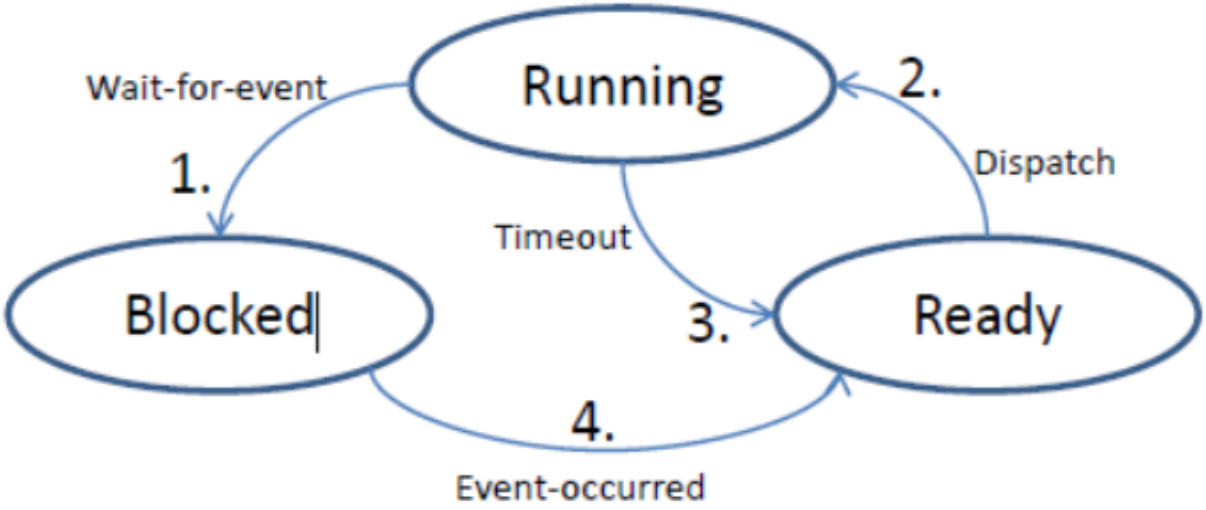
Threads vs Processes

- Less time to create a new thread (10-times faster than process creation in Unix)
- Less time to terminate a thread
- Less time to switch between threads
- Threads enhance efficiency in communication between different communicating programs, no need to call kernel routines, as all threads live in same process context

Threads vs. Processes

- ***Advantages of Threads***
  - Much faster to create a thread than a process
    - Spawning a new thread only involves allocating a new stack and a new thread control block
  - Much faster to switch between threads than to switch between processes
  - Threads share data easily
- ***Disadvantages***
  - Processes are more flexible
    - They don't have to run on the same processor
  - No protection between threads
    - Share same memory, may interfere with each other
  - If threads are implemented as user threads instead of kernel threads
    - If one thread blocks, all threads in process block

Thread States



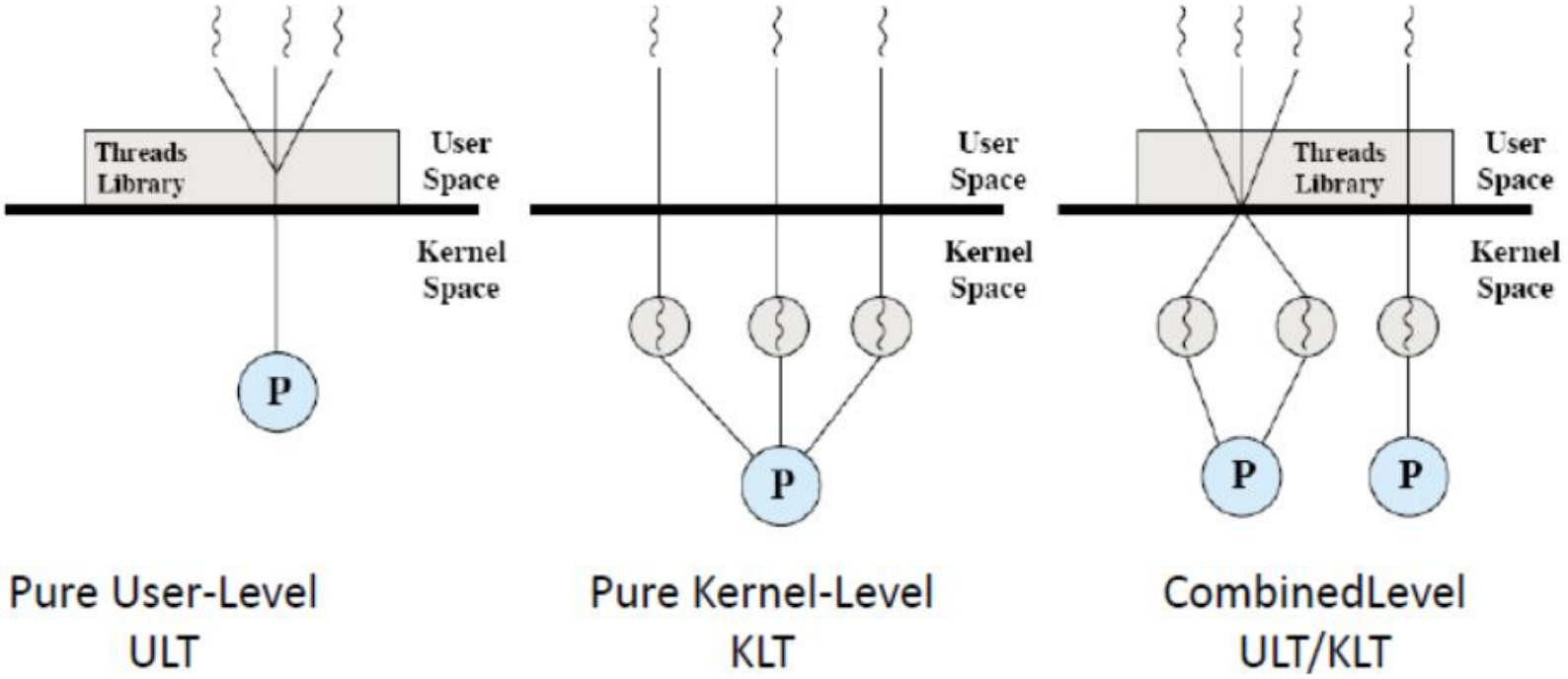
- Threads have three states
  - Running: CPU executes thread
  - Ready: thread control block is placed in Ready queue
  - Blocked: thread awaits event

Thread Operations

- There are four basic operation for managing threads
  - Spawn / create
    - A thread is created and provided with its own register context and stack space, it can spawn further threads
    - It is placed on the Ready queue
  - Block:
    - if a thread waits for an event, it will block and placed on the event queue
    - the processor may switch to another thread in the same or a different process
  - Unblock:
    - When the event occurs, for which the thread is waiting, it is placed on the ready queue
  - Finish:
    - When a thread completes, its register context and stacks are de-allocated

Thread Implementation

- Two main categories of thread implementation
  - User-level Threads (ULTs)
  - Kernel-level Threads (KLTs)Block:



Pure User-Level ULT

Pure Kernel-Level KLT

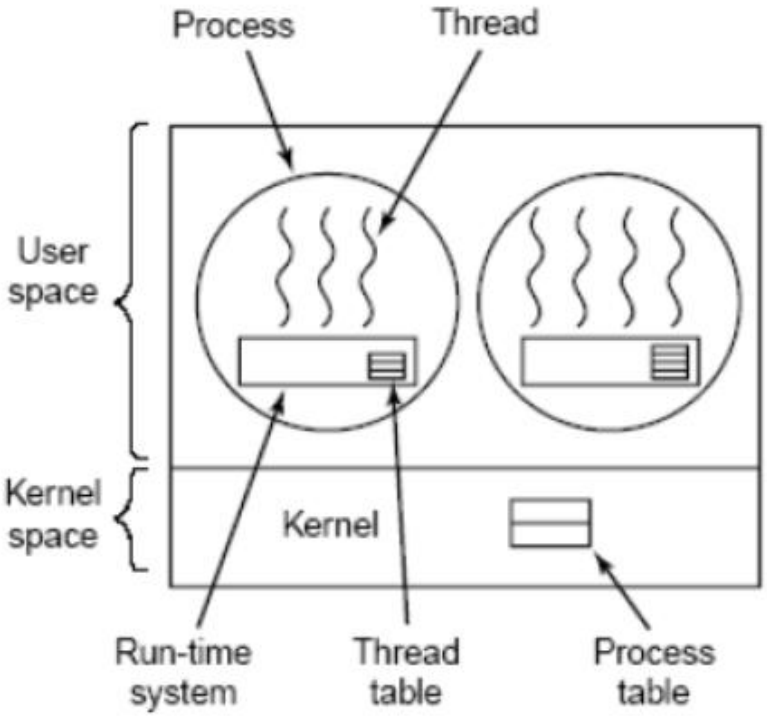
CombinedLevel ULT/KLT

User-Level Threads

- User-Level Threads
  - Threads are managed by application, using thread library functions
  - Kernel not aware of the existence of threads, it knows only processes with one thread of execution
- Benefit
  - Light thread switching
    - Does not need kernel mode privileges
- Also called “green threads” on some systems (e.g. Solaris)

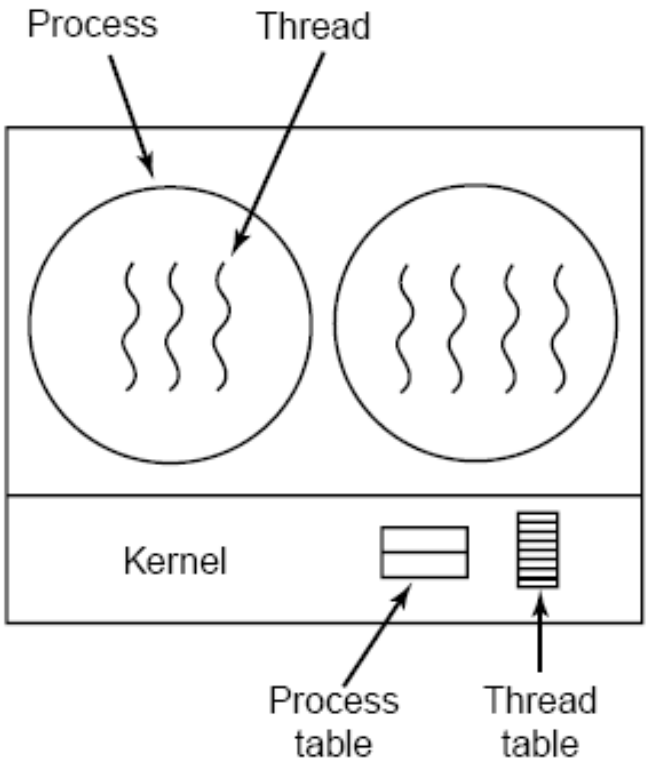
User-Level Threads

- Problem
  - If one thread blocks, the entire process is blocked, including all other threads in it
  - Only one thread can access the kernel at a time, as the process is the unit of execution known by kernel
  - All threads have to run on the same processor in a multiprocessor system
    - cannot run in parallel utilizing different processors at the same time, as process is dispatched on one



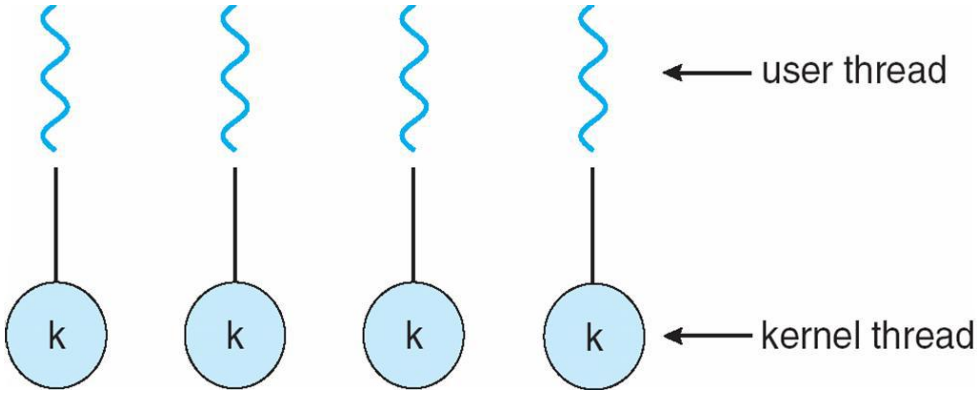
Kernel-Level Threads

- Threads are managed by Kernel
  - Creating / terminating threads via system calls
- Benefit
  - Fine-grain scheduling done on thread basis
  - If a thread blocks (e.g. waiting for I/O), another one can be scheduled by kernel without blocking the whole process
  - Threads can be distributed to multiple processors and run in parallel
- Problem
  - Heavy thread switching involving mode switch



One-to-One Model

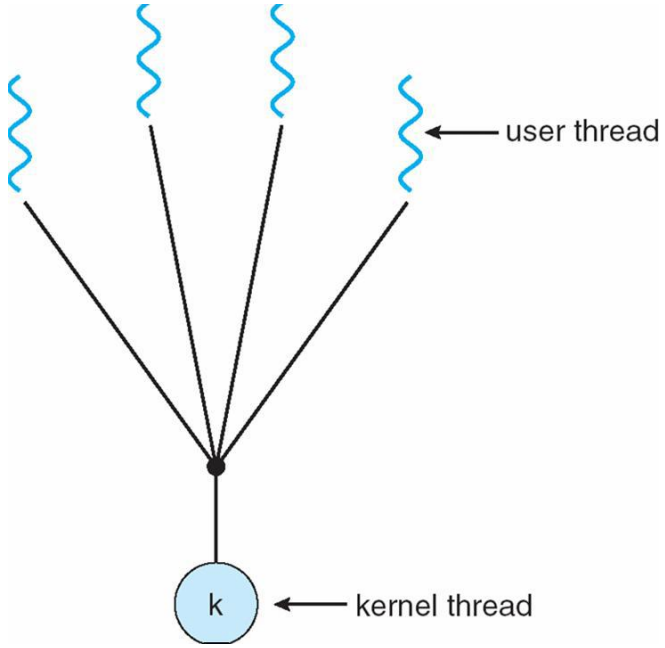
- Each user-level thread mapped to a kernel thread
- One blocking thread does not block other threads
- Multiple threads access kernel concurrently Benefit



- Problem
  - Creating a user-level thread requires creation of corresponding kernel thread
  - Kernel may restrict the number of threads created
- Example systems
  - Windows,

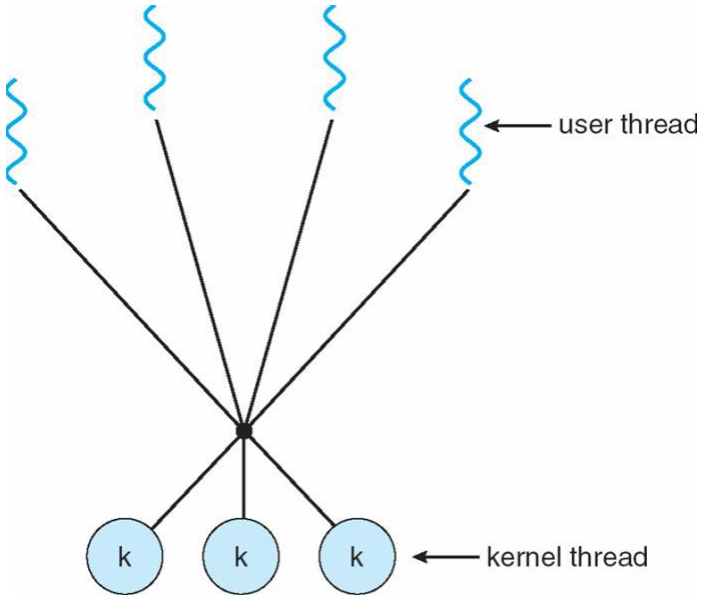
Many-to-One Model

- All user-level threads of one process mapped to a single kernel-level thread
- Thread management in user space
  - Efficient
  - Application can run its own scheduler implementation
- One thread can access the kernel at a time
  - Limited concurrency, limited parallelism
- Examples
  - “Green threads” (e.g. Solaris)
  - Gnu Portable Threads



Many-to-Many Model

- Many user-level threads are multiplexed (mapped dynamically) to a smaller or equal number of kernel threads
  - No fixed binding between a user and a kernel thread
- The number of kernel threads is specific to a particular application or computer system
  - Application may be allocated more kernel threads on a multiprocessor architecture as on a single-processor architecture
- No restriction on user-level threads
  - Applications can be designed with as many user-level threads as needed
  - Threads are then mapped dynamically onto a smaller set of currently available kernel threads for execution



Two-level Model

- Is a variant of the Many-to-Many model, allows a fixed relationship between a user thread and a kernel thread
- Was used in older Unix-like systems
  - IRIX, HP-UX, True64 Unix, Solaris 8

